

Conversational Liquidity-Pool Execution

End-to-End Logical Specification for IlyonAi

Product	IlyonAi — chat-based, non-custodial DeFi assistant (ilyonai.com)
Scope	17 EVM chains + Solana mainnet · 40+ protocol families · all pool primitives (V2/V3/V4, CLMM/DLMM, ALM, stable, weighted, vault, LST/LRT)
Posture	Green-field conceptual + logical design · LLM never produces calldata · every step user-signed
Optimized for	2026-05 protocol landscape · single-chat composed cross-chain plans · async-fill aware
Doc version	v1.0 · 2026-05-14

This document is the conceptual + logical contract between intent and on-chain state for every LP action IlyonAi will execute. It does not contain code. It is the specification by which the engineering team builds, tests, and audits chat-based pool execution end-to-end.

Table of Contents

1. Executive Summary
2. Glossary & Conventions
3. The Five-Stage Conceptual Pipeline
4. Logical Architecture: Nine Services Behind One Chat Box
5. Canonical State Machine (Mermaid + walkthrough)
6. The Seven Head-On Issues
 - 6a · Aerodrome Slipstream & Uniswap V4 native exec
 - 6b · Solana CLMM / DLMM range UI in chat
 - 6c · Cross-chain composed plans (ETH-USDT → Solana JLP)
 - 6d · "With my USDT" silent reassignment
 - 6e · APR-by-range curve from real data
 - 6f · Stuck-balance recovery
 - 6g · Receipt-token verification
7. User-Funding Scenario Matrix (15 scenarios)
8. Protocol-Family Matrix (master table)
9. Per-Family Deep Dives (weighted depth)
 - 9a · Uniswap V3 9h · Raydium AMM / CPMM / CLMM
 - 9b · Uniswap V4 9i · Orca Whirlpools
 - 9c · PancakeSwap V3 9j · Meteora DLMM / DAMM v2
 - 9d · Aerodrome Slipstream / Velodrome CL 9k · Kamino Vaults + Klend
 - 9e · Curve (legacy, NG, twocrypto-ng) 9l · JLP / JitoSOL / Marinade hub
 - 9f · Balancer V2 + V3 9m · LST / LRT hub set
 - 9g · Aave V3 + Pendle V2
10. Stub Families (delegate-to-aggregator)
11. Technical-Surface Recommendations & Trade-offs
12. Normalized JSON Schema for the Deposit Plan
13. Edge-Case Appendix
14. Open Design Questions

1. Executive Summary

IlyonAi today executes swaps, bridges, staking, and rudimentary LP actions through a single chat interface, backed by Enso (EVM), Jupiter (Solana), deBridge (cross-chain), and a curated data plane (DefiLlama, Moralis, Helius, Birdeye). The gap is **conversational liquidity-pool execution** — the user types an intent like “*add liquidity to USDC/SOL with my USDT*” and expects the assistant to return a small, ordered set of signable transactions plus an interactive yield card with range presets and APR projections, with no protocol UI ever visited.

This document is the green-field, end-to-end logical specification for that surface. It defines (a) the five-stage conceptual pipeline every LP action passes through, (b) the nine-service architecture behind the chat box, (c) a canonical typed *LiquidityIntent* + *DepositPlan* schema that decouples the LLM from calldata, (d) per-protocol-family deposit semantics for every major pool primitive on the eighteen chains in scope, (e) the cross-chain async-fill architecture for composed plans, (f) the seven specific issues the team has hit (Slipstream factory keying, V4 PoolKey, Solana CLMM/DLMM in-chat range, “with-my-USDT” reassignment, real-data APR curves, stuck-balance recovery, receipt verification), and (g) the exhaustive edge-case appendix.

The product thesis is one chat, signed steps. Two-separate-plans is a regression. Cross-chain compositions snapshot the deBridge solver-quoted destination amount, emit the deposit step with a *pending_dst_fill* blocker, then rebuild post-fill from the actual delta — recomputing optimal range-ratio leg amounts to absorb 5–20 bps of solver slippage. The same pattern handles any async leg: Pendle epoch entry, Lido stake queue, Solana confirmation lag.

The non-negotiable safety invariants are eight rules. The LLM never emits calldata, addresses, signatures, or amounts. Every broadcast is preceded by a fresh simulation no older than 30 seconds. Unlimited approvals are disabled by default — Permit2 or scoped allowances only. On-chain strings are sanitized before reaching the planner (defense against prompt-injection via token names). Critical limits are mirrored on-chain for any session-key flow. Every action has a one-click revoke. State drift between confirmation and broadcast triggers automatic re-simulation. Every prompt-to-tx pair is logged in a cryptographically signed audit trail.

Versioning posture is parallel paths. V1 is the today-default; v2/v3 (Balancer V3, Uniswap V4, Curve stable-ng, Raydium CPMM, DAMM v2) ships behind feature flags with a side-by-side diff documented per family. This avoids being blinded by migration deadlines while not exposing users to immature ABIs by default.

How to read this document

This spec answers the canonical example in the brief — “*user has USDT, wants USDC/SOL pool, returns two signable transactions*” — and generalizes it to every realistic {chain, protocol, pool format, funding scenario, position type} tuple. Sections 6d, 7, 9h-9j, and 12 collectively define exactly what the assistant should emit for that case.

2. Glossary & Conventions

Used consistently throughout the doc. Cross-referenced when first introduced.

Term	Definition
AMM v2 / V2-style	Constant-product pool ($x \cdot y = k$) with full-range LP ERC-20 receipts. Uniswap V2, Sushi V2, PancakeSwap V2, Aerodrome V1, Velodrome V1, Camelot V2, Spookyswap, Spiritswap, Baseswap.
CL / CLMM	Concentrated liquidity. Uniswap V3-class. Positions live in a tick range; receipt is an NFT (EVM) or position-PDA (Solana). Capital efficiency $\gg$ V2, but require active management.
DLMM	Discretized concentrated liquidity in fixed-price <i>bins</i> (no swap fee inside a bin). Meteora DLMM, Trader Joe V2 LB. Bin step is the parameter; <i>active_id</i> is the current bin.
ALM	Automated Liquidity Manager. Vault wrapper around a CL pool that auto-rebalances ranges. Arrakis, Gamma, Steer, Bunni, Krystal, Ichi, Beefy CL.
LST / LRT	Liquid Staking Token / Liquid Restaking Token. Receipt-bearing wrappers around staked ETH (stETH, rETH, sfrxETH, ETHx, mETH) or restaked variants (eETH/weETH, ezETH, rsETH, rswETH, pufETH).
Permit2	Uniswap's universal approval contract. One signed permit covers multiple downstream spenders. Mandatory in Uniswap V4, Balancer V3, UniswapX, most modern routers.
PoolKey	Uniswap V4 pool identifier: (<i>currency0</i> , <i>currency1</i> , <i>fee</i> , <i>tickSpacing</i> , <i>hooks</i>). The <i>poolId</i> is <code>keccak256(abi.encode(PoolKey))</code> . Pools share storage in a singleton PoolManager — no per-pool contracts.
Position-NFT Position-PDA	vs EVM CLMM receipts are ERC-721 NFTs (Uniswap V3 NFPM, V4 PositionManager, Slipstream NFPM). Solana CLMM/DLMM receipts are Program-Derived Accounts owned by the user's wallet.
Tick / Bin	Discrete price grid. Tick spacing is per-pool (V3 uses 1, 10, 60, 200; Slipstream 1, 50, 100, 200, 2000). DLMM bin step is in basis points (1 bp, 5 bp, 20 bp, 100 bp typical).
Receipt token	What the user holds after deposit. LP ERC-20 (V2/Curve/Balancer pre-V3), BPT, NFT (V3-family), vault share (ERC-4626), kToken, position-PDA. Used by the assistant for verification.
Composed plan	A multi-leg plan that crosses chains or protocols and is presented as a single, coherent flow in chat. Async legs carry <i>pending_dst_fill</i> blockers that resolve on fill webhooks or RPC polling.
blocker	A state on a step in the DepositPlan that prevents promotion to <i>ready</i> . Examples: <i>pending_dst_fill</i> , <i>adapter_quote_required</i> , <i>price_drift_resimulate</i> , <i>approval_missing</i> , <i>ata_creation_missing</i> .

Conventions. Default slippage 50 bps (user-overridable 10–500). Default deadline 600s. Public mempool default; private RPC flagged where mandatory. Position sizes \$50–\$50,000. Receipt verification required within 60 seconds of broadcast, surfaced in chat. Cross-chain default = deBridge DLN, fallback LI.FI / Socket, never raw burn-and-mint native bridges unless explicitly justified.

3. The Five-Stage Conceptual Pipeline

Every LP action — regardless of protocol, chain, or pool format — resolves to the same five-stage pipeline. Users may enter at any stage (paste a pool URL → skip Resolution; click an alert → enter at Preview). The pipeline is unidirectional for safety but reversible by user intent: any backward jump invalidates the simulation artifact and forces a refresh.

3.1 Stage 1 — Intent

The Planner (LLM under JSON-schema constrained decoding via OpenRouter → Claude / GPT / Gemini / Grok) extracts a partial *LiquidityIntent* from the natural-language prompt. Outputs are typed structures and clarification questions — never calldata, addresses, or amounts that weren't in the prompt.

Fields extracted (with confidence + source span):

- **action:** ADD · INCREASE · DECREASE · COLLECT · REBALANCE · CLOSE · ZAP_IN · ZAP_OUT · MIGRATE
- **chain:** explicit (“on Base”) or inferred from token symbol (“SOL” → Solana) or asked
- **protocol:** explicit (“on Uniswap V3”) or null (resolved in Stage 2 by ranking)
- **pair:** token0 / token1 symbols (case-insensitive, alias-aware: ETH↔WETH, MATIC↔POL)
- **fee_tier / tick_spacing / bin_step:** if user said “1 bp pool” or “0.05% tier”
- **amounts:** EXACT_USD · EXACT_TOKEN0 · EXACT_TOKEN1 · EXACT_BOTH · PROPORTIONAL · PERCENT_OF_POSITION · ALL
- **source token / source chain:** “with my USDT” → source override (see §6d)
- **range:** FULL · WIDE · BALANCED · TIGHT · CUSTOM_TICKS · CUSTOM_PRICE or null
- **strategy:** SPOT · CURVE · BID_ASK (DLMM); MAVERICK_STATIC / RIGHT / LEFT / BOTH
- **execution overrides:** slippage, deadline, MEV protection, stake-rewards toggle

Unambiguous fields: action verb, native amount with explicit unit (“0.5 ETH”), explicit pool address, explicit chain name. **Always needs a clarifier:** ambiguous token symbol (USDC on which chain?), missing amount, ambiguous range (“narrow”). **Auto-resolvable with disclosure:** protocol omitted → rank by TVL × volume × age, show top 3 + “show all”; fee tier omitted → recommend per pair-category; range omitted → Balanced default with explanation.

3.2 Stage 2 — Resolution

Deterministic, never LLM-driven. The Entity Resolver maps fuzzy entities to canonical tuples. The Pool Index returns ranked candidate pools.

Resolution steps in order:

- **Token resolution:** symbol → (chain_id, contract_address, decimals). Verified against Ilyon’s curated token list + on-chain symbol() and decimals() reads. Ambiguous symbols (USDC exists on every chain; SOL exists native + as Wormhole-wrapped on EVM) return ranked candidates by TVL × user-balance-on-chain × default-chain-preference.
- **Pool ranking:** query the Pool Index (daily-refreshed cache of every protocol’s subgraph or program-state index). Rank by: $weighted_score = 0.4 \cdot \log(TVL) + 0.3 \cdot \log(7d_volume) + 0.15 \cdot fee_apr_30d + 0.1 \cdot pool_age_days/365 + 0.05 \cdot audit_status$. Filter by Shield allowlist (top-N per chain per protocol unless user opted into experimental).

- **Protocol family head selection:** when user says “Raydium pool” without specifying AMM vs CPMM vs CLMM, default to the one with highest TVL for that pair. Same for Orca v1 vs Whirlpool, Meteora dynamic vs DLMM vs DAMM v2, Uniswap V2 vs V3 vs V4.
- **Fee-tier resolution:** per pair-category defaults — stable/stable → lowest available (1 bp / 5 bp); blue-chip/stable → 5 bp; blue-chip/blue-chip → 5 bp or 30 bp by volatility; exotic/blue-chip → 30 bp; exotic/exotic → 100 bp.
- **Tick spacing constraint:** Slipstream and V4 are keyed by tickSpacing not fee_bps (§6a). Resolver returns a (poolAddress, tickSpacing, currentTick, sqrtPriceX96) tuple and rounds any user-stated range to the nearest valid ticks at preview time.

3.3 Stage 3 — Sizing & Range

Quantitative parameters: amounts, slippage, range bounds. Assistant proposes defaults; user overrides with a slider, preset button, or text response.

- **Amount mode:** if user gave USD, convert via Pyth/Chainlink price; if user gave one token, compute the optimal counter-token amount for the chosen range (V3: from *LiquidityAmounts.getLiquidityForAmount0/1*; DLMM: from bin distribution; V2: 1:1 ratio).
- **Range presets** for CL pools: **Full** (1× capital efficiency, IL = HODL diff, always in range); **Wide ±25%** (~4× efficiency, ~95% in-range historically for blue-chips); **Balanced ±10%** (~10× efficiency, ~78% in-range); **Tight ±5%** (~20× efficiency, ~52% in-range, expect weekly rebalance). Stable pairs use ±0.5% / ±0.1% / ±0.05%. See §6e for the real-data APR curve.
- **Slippage budget:** 50 bps default, distributed across legs. For a USDT → USDC + SOL zap-in, slippage is split: 15 bps each on the two swap legs + 20 bps headroom on the deposit ratio enforcement. Stable pairs default to 10 bps total.
- **Dust accumulation:** each leg leaves rounding remainders. Track per-leg dust; if cumulative dust > \$1, offer to either (a) sweep into one of the position tokens via a tiny final swap, (b) leave in wallet, or (c) increase position by dust amount on next rebalance. Default: leave in wallet, show in confirmation card.

3.4 Stage 4 — Preview

Full bundle simulation via Tenderly (EVM) or simulateTransaction (Solana). The Preview card is the user's single source of truth before signing. Required panels:

- **Assets out:** token, amount, USD value, source-of-funds breakdown (which wallet balance is being spent, any swap/bridge legs in between).
- **Receiving:** receipt token type (NFT id, LP ERC-20 amount, position-PDA address, kToken share), expected liquidity L, range bounds in price + tick, expected fee APR with source (*DefiLlama 30-day*), expected reward APR with token breakdown.
- **Gas:** per-leg gas cost, total in USD, gas-token availability check (auto-flag if user is missing ETH on dst chain and offer to bundle a tiny ETH bridge first).
- **Risk panel:** Shield verdict on each token, allowlist status, MEV exposure flag, expected price impact, JIT-attack risk, IL projection table (5 price-move scenarios), tick-spacing rounding diff (“you asked for \$3,795–\$4,602, valid range is \$3,800.12–\$4,599.88”).
- **Toggles:** stake-rewards (auto-deposit into gauge / masterChef / Slipstream CLGauge), MEV protection (Flashbots / MEVBlocker / private RPC), permit2 vs approve.
- **One Confirm button:** produces an artifact hash; the wallet popup's calldata must hash-match this artifact, or signing is refused.

3.5 Stage 5 — Execute & Track

The Signer Orchestrator submits via the chosen route, surfaces a live progress card with tx_hash and block-explorer link, and on confirmation enrolls the position with the Indexer.

- **Per-step state:** queued → broadcasted → confirmed (with reorg-protection by waiting N blocks per chain) → verified (receipt-token RPC check, §6g) → indexed.
- **Async-fill rebuild:** if a bridge leg is in-flight, the deposit step sits in *pending_dst_fill*. On fill (webhook from deBridge DLN, or RPC poll of dst-chain balance delta), the Builder re-computes optimal leg amounts with the actual filled amount and promotes to ready. User signs the deposit only after fill.
- **Lifecycle handoff:** position is enrolled with the Indexer; alerts fire on out-of-range, fee-APR drops, approaching boundary, TVL exodus, token re-scan flags, gas-favorable rebalance windows.

4. Logical Architecture: Nine Services Behind One Chat Box

The chat surface is intentionally thin. Behind it sit nine services with narrow contracts, sequenced by a stateful workflow engine. This separation is what makes the system safe under prompt-injection and LLM hallucination — the LLM may plan and talk; it may not produce calldata, addresses, amounts, or signatures.

Service	Responsibility
Planner	LLM + tool use under JSON-schema constrained decoding. Routes via OpenRouter (Claude / GPT / Gemini / Grok). Inputs: prompt + Resolver replies. Outputs: typed LiquidityIntent (partial OK) + clarification questions. Never emits bytes.
Entity Resolver	Deterministic. Maps symbols → (chain_id, address, decimals) via Ilyon's curated list + on-chain verification. Returns ranked candidates on ambiguity; never silently resolves.
Pool Index	Daily-refreshed cache from each protocol's subgraph (EVM) or program-state index (Solana) + DefiLlama Yields + Merkl rewards. Schema includes pool address, tokens, fee tier, tickSpacing, TVL, 7d volume, fee APR, reward APR, age, audit status.
Sizing & Range Engine	Protocol-native math: LiquidityAmounts for V3-family, bin distributions for DLMM/Maverick, balanced weights for Balancer, Curve invariants. Range presets driven by 7-day realized volatility. Outputs proposed (lower_tick, upper_tick, amount0, amount1, expected_L, expected_fee_apr, expected_in_range_prob).
Calldata Builder	Per-protocol adapter library. Uses canonical SDKs where mature (@uniswap/v3-sdk, @uniswap/v4-sdk, @orca-so/whirlpools, @raydium-io/raydium-sdk-v2, @meteora-ag/dlmm, Curve metaregistry, Balancer SDK V2/V3). Raw ABI/IX encoding only for SDK gaps (Slipstream, Maverick V2, some Algebra forks).
Simulator	Tenderly bundle simulation for EVM (with eth_call + state-override fallback for cost-sensitive paths); Solana simulateTransaction with replaceRecentBlockhash for SVM. Returns asset-diff, gas, expected position state, decoded events, revert reason.
Shield Gate	Extends Ilyon's Protect product. Token risk (GoPlus + Blockaid), pool allowlist, honeypot / depeg / TVL-stability, OFAC screening, V4 hook allowlist, fee-on-transfer detection, sandwich-MEV exposure, JIT-attack adjacency to large pending swaps.
Signer Orchestrator	Wallet handshake (Phantom + Solflare + WalletConnect for Solana; MetaMask + WalletConnect for EVM; Phantom for EVM where supported). Enforces simulated_calldata_hash == broadcast_calldata_hash. Re-simulates if >30s elapsed.
Indexer + Notifier	Per-position subscriptions. PositionHealth snapshots every 5 min: current value, HODL value, fees collected/uncollected, IL %, in-range bool, time-in-range, realized fee APR. Notifier fires user-configurable alerts back into chat.

All nine services share one **State Store** keyed by *intent_id*, persisting {user_id, raw_prompt, parsed_intent, missing_fields, resolved_entities, simulation_artifact, signature, tx_hash, position_id}. State is required because LP intents are multi-turn (clarify range → re-simulate after price drift) and because the audit trail demands a tamper-evident link from prompt to on-chain hash.

5. Canonical State Machine

The lifecycle from chat message to verified position. The diagram below is the authoritative state machine; every implementation detail must be expressible as a transition in this graph.

Mermaid source:

```
stateDiagram-v2
    [*] --> Prompted
    Prompted --> Parsing : LLM constrained-decode
    Parsing --> Clarifying : missing_fields not empty
    Clarifying --> Parsing : user answers
    Parsing --> Resolving : intent typed
    Resolving --> Clarifying : ambiguous symbol/pool
    Resolving --> Sizing : pool + tokens resolved
    Sizing --> Previewing : amounts + range set
    Previewing --> Sizing : user edits range/amount
    Previewing --> Shielding : preview rendered
    Shielding --> Blocked : Shield refuses (rug, sanctions, depeg)
    Shielding --> Simulating : checks pass
    Simulating --> Previewing : simulation revert -> show reason
    Simulating --> ReadyToSign : sim OK, artifact_hash issued
    ReadyToSign --> ReSimulating : >30s elapsed before sign
    ReSimulating --> ReadyToSign : ReadyToSign
    ReadyToSign --> Signing : user clicks Confirm
    Signing --> Broadcasted : wallet returns signature
    Signing --> Cancelled : user rejects in wallet
    Broadcasted --> Confirming : tx_hash in mempool
    Confirming --> Verifying : confirmed N blocks
    Confirming --> StuckRecovery : timeout / underpriced gas
    StuckRecovery --> Broadcasted : RBF accepted
    StuckRecovery --> Cancelled : user cancels
    Verifying --> Indexed : receipt token verified
    Verifying --> Failed : execution-fail (slippage breach etc.)
    Failed --> StuckBalanceFlow : intermediate leg succeeded
    StuckBalanceFlow --> Resolving : auto-rebuild path
    StuckBalanceFlow --> Refunding : user picks refund-swap-back
    Indexed --> PendingDstFill : [*] state
    PendingDstFill <--> Broadcasted
    PendingDstFill --> PendingDstFill : was cross-chain bridge leg
    PendingDstFill --> Rebuilding : fill webhook OR rpc-poll delta
    Rebuilding --> Previewing : deposit step rebuilt with actual amount
```

Key transitions worth calling out:

- **Clarifying** ↔ **Parsing** is the only place the LLM re-engages; once the intent is typed, the LLM is read-only for the rest of the flow.
- **ReSimulating** fires on the 30-second drift rule. This is critical for V3-class pools where a 1% price move during a slow signer flow can move the optimal token ratio enough to either revert the mint or produce a one-sided deposit.
- **PendingDstFill** is the choice node that distinguishes a same-chain plan from a composed cross-chain plan. The deposit step blocks here, the chat shows a live fill-progress card, and the Builder rebuilds amounts post-fill (§6c).
- **StuckBalanceFlow** is the post-mortem branch when leg N succeeded but leg N+1 failed and an intermediate token is sitting in the wallet. Recovery posture is defined in §6f.

6. The Seven Head-On Issues

6a. Aerodrome Slipstream & Uniswap V4 native execution

Problem. Standard V3-style resolvers key the factory by *(tokenA, tokenB, fee_bps)*. Aerodrome Slipstream and Velodrome CL fork from Uniswap V3 but the factory is keyed by *tickSpacing*, not *fee*. Uniswap V4 doesn't have a factory at all — pools live inside a singleton PoolManager and are addressed by the hash of a five-field PoolKey.

Slipstream / Velodrome CL — correct resolver

The factory call is *getPool(tokenA, tokenB, tickSpacing) → poolAddress*. Common tickSpacings on Base are 1 (sub-bp stable pairs like USDC/USDbC), 50 (5 bp tier — most ETH/USDC volume), 100 (10 bp), 200 (30 bp), 2000 (1% exotic). Resolver must:

- Query the Slipstream factory's *tickSpacings()* list at boot, cache per chain.
- For each (tokenA, tokenB) requested, iterate over tickSpacings, return the set of (poolAddress, tickSpacing, fee, currentTick, sqrtPriceX96, TVL, 7d-volume) tuples.
- Display in chat as “tier-equivalent” labels (5 bp / 10 bp / 30 bp / 1%) but carry tickSpacing internally — never round-trip to fee for resolver calls.
- Mint flow is identical to V3's NonfungiblePositionManager — same NFPM ABI, just different deployment. Position NFT lives on the Slipstream NFPM, not Uniswap's.
- **Reward awareness:** Slipstream positions can be transferred to *CLGauge.deposit(tokenId)* to earn AERO emissions, but this **forfeits swap fees to veAERO voters**. Surface this trade-off explicitly in Preview as a toggle, defaulted OFF — it is not a neutral choice. Quote both APR streams separately.

Uniswap V4 — correct resolver and mint flow

V4 has no factory and no per-pool contracts. The PoolKey struct is *(currency0, currency1, fee, tickSpacing, hooks)* and the poolId is *keccak256(abi.encode(PoolKey))*. Resolver flow:

- Maintain a V4 pool index built from PoolManager *Initialize* events. Each event emits the full PoolKey plus initial sqrtPriceX96 and tick.
- For a (tokenA, tokenB) query, return all matching PoolKeys ranked by TVL × volume × **hook-allowlist status**. Hooks introduce arbitrary callback logic — Shield must refuse unknown hook addresses unless the user has explicitly opted into experimental pools.
- Native ETH support: V4 supports native currency by setting *currency0 = address(0)*. The Builder must not wrap to WETH for V4 pools that use native — V4 handles it.
- Mint flow goes through the periphery **PositionManager.modifyLiquidities(unlockData, deadline)**. The *unlockData* is an encoded action sequence: typically [MINT_POSITION, SETTLE_PAIR, TAKE_PAIR] — mint the position, settle the two currencies the user owes, take back any delta. The Builder constructs this sequence; the LLM never touches it.
- **Dynamic fees:** some hooks set the fee dynamically per swap. The Preview card must show an APR *range* (min/max from 30d hook history) rather than a single number for dynamic-fee pools, and explicitly flag dynamic fees as a property.

UX implication

Both Slipstream and V4 also break the user-facing mental model of “0.05% tier” because the tier no longer uniquely identifies a pool. The chat **MUST** present human-readable labels (“5 bp tier · 1-tick spacing · no hooks”) and carry the canonical pool identifier (address for Slipstream, poolId for V4) under the hood. Never let the user pick by tier alone for these protocols.

6b. Solana CLMM / DLMM range UI in chat

Problem. The current product degrades CL on Solana to “Jupiter-swap to one leg, then redirect to protocol dApp”. The right behavior is identical to V3-class on EVM: a draggable range selector in chat, plus presets, plus a real-data APR curve.

Whirlpool — open_position flow

Uses the modern `@orca-so/whirlpools` SDK. Conceptual instruction sequence:

- Pre-flight: ensure both token ATAs exist (create if missing, ~2,039 lamports each); ensure WSOL is wrapped + sync-native if SOL-side; ensure required TickArray accounts are initialized for the chosen range (each TickArray covers 88 ticks at the pool's tickSpacing; if user's range crosses uninitialized arrays, prepend `initialize_tick_array` IXs).
- **open_position_with_metadata** mints the position NFT (Metaplex metadata) to a user-owned position-mint, derives the position-PDA from the mint, and stores tick bounds.
- **increase_liquidity** deposits the two token amounts into the position, given an *liquidity_amount* calculated client-side from the chosen range and token amounts.
- If user opted in to `open_position_2022` (Token-2022 path with transfer-hook-aware tokens), use the corresponding instruction variant; the SDK exposes both.
- Bundle target: one signed transaction containing `approve_or_create_ATAs` + `sync_native` + (optional) `initialize_tick_array` × N + `open_position_with_metadata` + `increase_liquidity`. If size exceeds the 1232-byte tx limit, use an Address Lookup Table (ALT). Phantom mobile rejects ALTs with >~30 keys — split into two signed txs in that case (open + increase).

Raydium CLMM — openPositionFromBase flow

Uses `@raydium-io/raydium-sdk-v2`. Conceptual sequence:

- Pre-flight: ATAs + WSOL wrap. Compute (tickLower, tickUpper) rounded to pool tickSpacing.
- **openPositionFromBase** or **openPositionFromLiquidity** — the former takes one side's token amount and computes the other; the latter takes a pre-computed L. Builder uses `openPositionFromBase` by default since chat input is amount-driven.
- Output: position-NFT mint + position-PDA. Reward emissions from Raydium's farm system are auto-accrued; harvest via `harvestAllRewards`.

Meteora DLMM — initializePositionAndAddLiquidityByStrategy

Uses `@meteora-ag/dlmm`. DLMM is bin-based, not tick-based — the conceptual model differs slightly:

- Pre-flight: compute (*minBinId*, *maxBinId*) from user-stated price range and the pool's *binStep* (in bps). Ensure *BinArray* accounts covering the range are initialized — if not, prepend init IXs (binArrays cover 70 bins each).
- **initializePositionAndAddLiquidityByStrategy** takes (*minBinId*, *maxBinId*, *strategyType* ∈ {Spot, Curve, BidAsk}, *totalXAmount*, *totalYAmount*). Strategy types map to different liquidity distributions across bins — Spot is uniform, Curve concentrates around *active_id*, BidAsk concentrates at the edges (for range orders).
- **autoFillYByStrategy** is the SDK helper that computes the optimal Y amount given X and the strategy — used when the user gave only one side's amount.
- Token-2022 with transfer hooks: DAMM v2 supports them; DLMM has been adding support pool-by-pool. Builder must check pool's *token_program* field and use the matching CPI variant.

In-chat range UI flow (all three)

The chat surface renders a yield card with: current price, the 30-day price band, four preset buttons (Tight / Balanced / Wide / Full), and a draggable lower-and-upper price slider. The APR-by-range curve underneath shows fee APR + reward APR as a function of range width (the curve is described in §6e). When the user adjusts the range, the Sizing Engine recomputes amount0/amount1 ratio on the fly and updates the “you need X SOL + Y USDC” line. The whole interaction is local until the user clicks Confirm — no on-chain calls are made during exploration.

Solana signing trade-off

Two-sign vs one-sign trade-off. Whirlpool can be one signed transaction if the user has both tokens pre-funded and tickArrays exist. If a swap leg is needed first (typical USDT→SOL+USDC), it becomes two signed txs: tx1 swap, tx2 deposit. Don't try to force one signature with a Jupiter route → Whirlpool deposit CPI; that path exceeds the 1232-byte limit on all non-trivial pools and is brittle. Two signs is the right default; show clear progress in chat between them.

6c. Cross-chain composed plans — ETH-USDT → Solana JLP

Problem. The user has USDT on Ethereum, wants to deposit into JLP (Jupiter Perps LP) on Solana. The bridge is async (deBridge DLN solver-fill, 30–90s typical). The destination amount is not known precisely at planning time.

Architecture: snapshot → block → rebuild → promote.

- **Planning:** request a deBridge DLN quote for USDT(ETH) → USDC(Solana). The quote returns *solver_estimate_dst_amount* with a ±5–20 bps slippage band. Snapshot this as *expected_filled_amount* in the bridge step.
- **Composition:** build the full plan as a single object with three steps: (1) *approve_dst USDT* on Ethereum (Permit2 preferred), (2) *bridge* via deBridge DLN order, (3) *deposit JLP* on Solana via Jupiter Perps LP program. Step 3 carries a *pending_dst_fill* blocker. The chat shows all three steps with their dependencies.
- **User signs steps 1 and 2 immediately.** Step 3 is greyed out with “Waiting for bridge fill — usually 30–90 seconds”.
- **Fill detection:** subscribe to deBridge DLN order-status webhook AND poll the destination wallet's USDC ATA balance with exponential backoff (starts at 2s, max 15s). The first signal wins.
- **Rebuild:** on fill, the Builder reads the actual delta (which may differ from the snapshot by solver slippage), recomputes the JLP deposit amount, refreshes the Preview card with the actual numbers, and promotes step 3 to ready. The user signs the final deposit. The whole experience is one chat thread; the user doesn't see a “new plan”.
- **Fallbacks:** if fill takes >5 minutes, surface a recovery card with options: (a) keep waiting (show DLN solver-network status), (b) cancel the bridge and refund via DLN's cancel flow, (c) deposit the bridged USDC into a different pool, (d) leave funds in wallet and notify when ready.

The same pattern reuses for any async leg: Pendle PT/YT mint (epoch boundary), Lido stake queue (sometimes minutes during congestion), Marinade direct-stake (next-epoch activation), Curve gauge boost (next-epoch checkpoint), even unusually slow Solana confirmations during periods of leader skip-rate spikes. Generalize the blocker as *pending*: { *kind*, *expected_resolve_ms*, *watch_strategy*: *webhook* | *rpc_poll* | *both* }.

6d. “With my USDT” silent reassignment

Problem. User says “USDC/SOL pool with my USDT”. The pool needs USDC + SOL; the user has USDT. Three possible behaviors:

- **(A) Silent split:** swap USDT → USDC + SOL in optimal ratio for the chosen range, show the result in Preview. **Pro:** minimum friction, mirrors what a smart trader would do. **Con:** silently spends two-thirds of the funds on the swap to SOL — user might not realize the SOL leg is half their capital and now exposed to ETH↔SOL volatility.
- **(B) Clarifier:** ask “Should I swap your USDT into both legs of the pool (default), deposit only the USDC side (single-sided LP), or pick a different pool that takes USDT?”
- **(C) Smart heuristic:** detect if a USDT-bearing alternative exists with comparable TVL/APR (e.g., USDT/SOL pool) and propose it alongside the original. If the alternative is within 20% APR and 50% TVL of the original, prefer it for simplicity.

Recommended default: (C), with (B) as fallback when no comparable alternative exists. Reasoning: zapping USDT into both legs is mechanically correct but quietly increases the user's risk exposure (now exposed to USDC↔SOL volatility on capital they explicitly didn't hold in either). A USDT-native alternative honors the user's asset allocation. If none exists, the clarifier ensures informed consent before the asymmetric exposure.

Preview card requirements when (A) or (C-fallback-to-A) executes: explicitly say “You said USDT. I'll split into 50.3% USDC + 49.7% SOL (the ratio for your selected range). After this trade, your position is exposed to SOL price movement, not just USDC peg.” Surface the implied USDT-to-SOL exposure as a notional dollar amount on the IL projection.

6e. APR-by-range curve from real data

Problem. The current product synthesizes APR with a Gaussian-tanh CDF over price ratios. That's defensible as a placeholder but produces inaccurate numbers — especially for stable pairs (over-promises) and exotic pairs (under-promises). Real data and proper math fix both.

Data sources, in preference order

- **On-chain swap event aggregation** (preferred for accuracy): for each pool, sum *fee_growth_inside_X* deltas per tick range over a rolling 30-day window. This is the ground truth for what fees a position in that range would have earned. Available via each protocol's subgraph (Uniswap, PancakeSwap, Slipstream, Curve) or program-state index (Whirlpool, Raydium CLMM, DLMM).
- **GeckoTerminal / Birdeye** for hourly OHLCV and 30d volume per pool — used to compute the realized fee yield as $fee_apr \approx (30d_volume \times fee_tier \times 365 / 30) / TVL$.
- **CoinGecko 30d hourly** for token price series — used to compute realized log-return volatility for in-range probability.
- **Pyth Network historical** for higher-resolution price (sub-hour) when needed for stable-pair tight ranges where hourly granularity loses signal.
- **Coinalyze / Hyperliquid perp funding** as a forward-looking volatility proxy — blends with realized vol on a 70/30 weight to reduce regime-shift lag.
- **Merk1 API** for external reward APRs (AERO, VELO, ARB, OP gauge incentives) — these are additive to fee APR and must be quoted separately because they're forfeitable under Slipstream's stake-vs-fees toggle.

Math — the defensible formula

Let p = current price, p_L , p_U = range bounds. Let σ = 30-day realized log-return volatility (annualized). The four factors:

1. **Capital efficiency** (V3 math, exact): $CE = 1 / (1 - \sqrt{(p_L / p)} \cdot \frac{1}{2} - \sqrt{(p / p_U)} \cdot \frac{1}{2})$
(Equivalent to $L_concentrated / L_full_range$ for the same dollar value.)

2. In-range probability (empirical CDF, not Gaussian): $P_{in} = (\text{count of hourly observations where } p_L \leq p_t \leq p_U \text{ over 30d}) / (\text{total hours})$.

Always compute from the empirical distribution, never assume log-normal — fat tails on exotic tokens routinely break the parametric assumption.

3. Realized fee yield at full range: $\text{fee_yield_full} = (30d_swap_volume \times \text{fee_tier}) / TVL \times (365 / 30)$.

4. IL drag at expected exit price (assume mean-reversion if stable pair, random walk otherwise): $IL_drag = \text{expected_IL_pct} \times \text{position_holding_period_yrs}$.

Final composed APR for a range: **APR(range) = CE × P_{in} × fee_yield_full + reward_APX – IL_drag**.

Curve construction: evaluate the formula for 40 range-width values from $\pm 0.05\%$ (tightest reasonable for stables) to $\pm 200\%$ (effectively full range), plot APR on Y, range width on X. The result is a hump curve — peaks at the width where $CE \times P_{in}$ is maximized, drops off at both tails (too tight = out-of-range; too wide = no concentration). Use this curve to drive the four preset recommendations: Tight = $0.6 \times$ peak-width, Balanced = $1.0 \times$ peak-width, Wide = $1.8 \times$ peak-width, Full = $\pm 200\%$.

6f. Stuck-balance recovery

Problem. Tx N of M succeeded; tx N+1 failed. User now has an intermediate token in their wallet (post-swap, pre-deposit). The assistant must take a clear posture.

Decision tree, evaluated top-down:

- **If the next leg failed because of slippage breach and the failure was <5 minutes ago:** auto-rebuild with a wider slippage tolerance (within the user's configured max) and offer one-click resume. No refund-swap-back.
- **If the next leg failed because the target pool no longer exists / was paused / hit a deposit cap:** surface alternatives ranked by APR + similarity to original intent. Do NOT auto-route — ask.
- **If the next leg is a cross-chain bridge that errored on submission:** hold the intermediate balance; offer alternative bridges (LI.FI, Socket); never auto-pick.
- **If the next leg was a deposit and the user explicitly cancelled in their wallet:** do nothing automatic. Surface the intermediate balance as a "stuck" item in chat with three buttons: *Resume deposit*, *Swap back to source*, *Leave in wallet*.
- **If >30 minutes have elapsed since the leg failed and the user hasn't returned:** send a notification (in-app + optional email/Telegram) but do not auto-act.

Hard rule: never auto-refund-swap-back without user consent. A refund swap is a second taxable event (in jurisdictions that tax crypto) and a second slippage cost. The only auto-action allowed is auto-rebuild with a wider slippage on the same intent — and only within the user's configured tolerance band.

6g. Receipt-token verification

Within 60 seconds of broadcast confirmation, the assistant must positively verify that the user received the expected receipt token. The verification RPC read depends on the family:

Protocol family	Receipt	Verification RPC read
Uniswap V2 / V2 forks	ERC-20 LP	<i>balanceOf(user, lpToken)</i> delta > expected_lp_amount × (1 – slippage)
Curve	ERC-20 LP	Same as V2; also <i>get_virtual_price()</i> sanity check (peg drift)
Balancer V2	BPT (ERC-20)	<i>balanceOf(user, bpt)</i> delta; <i>Vault.getPoolTokenInfo</i> ratio check
Balancer V3	BPT (ERC-20)	Same as V2; new singleton Vault, different read path
Uniswap V3 / PancakeSwap V3 / Slipstream / Velodrome CL	Position NFT	<i>NFPM.positions(tokenId)</i> returns liquidity > 0 AND tickLower/Upper match plan
Uniswap V4	Position NFT (V4 PositionManager)	<i>PoolManager.getPositionInfo(poolId, owner, salt)</i> shows liquidity > 0
Aave V3	aToken (ERC-20)	<i>balanceOf(user, aToken)</i> delta ≥ supply_amount; isolated-mode flag check
Pendle V2	PT + YT (ERC-20s) or LP (ERC-20)	Both PT and YT balances if mintPY; LP balance if addLiquidity
ERC-4626 vaults (Yearn V3, Beefy, Origin, Etherfi)	Vault share	<i>convertToAssets(balanceOf(user))</i> ≥ deposit × (1 – slippage)
Orca Whirlpools	Position-PDA + position-NFT	Position account exists, owner = user, liquidity > 0, ticks match plan
Raydium AMM v4	LP mint (SPL token)	ATA balance delta; note: not Jupiter-tradable, must redeem natively
Raydium CPMM	LP mint (SPL token)	ATA balance delta; redemption via CPMM program
Raydium CLMM	Position-NFT	Position account read; tick bounds match plan
Meteora DLMM	Position-PDA (no NFT)	Position account exists; bin distribution matches strategy chosen
Kamino Vaults	Strategy-share SPL token (kToken-like)	ATA balance delta
Kamino Lend (klend)	Obligation account state	Obligation deposit_amount field updated
JLP	JLP SPL token	ATA balance delta; <i>Jupiter Perps</i> program-state confirms LP mint
Marinade / JitoSOL / sanctum INF	mSOL / JitoSOL / INF (SPL)	ATA balance delta; exchange rate sanity vs oracle
LST hubs (Lido stETH, Rocket rETH, Frax sfrxETH, etc.)	Receipt ERC-20	<i>balanceOf</i> delta; for Lido, also <i>getPooledEthByShares</i> sanity
LRT hubs (eETH, ezETH, rsETH, rswETH, pufETH)	Receipt ERC-20	<i>balanceOf</i> delta; restaking state read for queued portions

Verification UX during the 60s window. Chat surfaces a live status card: “Broadcasting → Confirmed (block #X) → Verifying receipt...”. If verification succeeds, the card flips to “Position confirmed” with the receipt token details. If verification fails after 60s, the card flips to “Verification timed out — your tx confirmed but the receipt token isn’t showing yet. This sometimes means the indexer is lagging. [Check on explorer] [Retry verify] [Notify me when ready]”.

7. User-Funding Scenario Matrix

Fifteen realistic funding scenarios with their canonical ordered plan. Each scenario is a row in the test matrix; the assistant must handle all of them deterministically.

# Scenario	Concrete example	Ordered plan	Notes
1. Same-chain, both tokens already held	USDC + ETH on Base, deposit to Aerodrome Slipstream USDC/WETH 5bp	(1) approve token0 [Permit2] · (2) approve token1 [Permit2] · (3) mint	Single signature with Permit2 batch; otherwise three signs
2. Same-chain, single source token	\$2k USDC on Base → ETH/USDC 5bp Slipstream	(1) split-swap USDC → ETH (50% via Enso) · (2) approve · (3) mint	“Half your USDC will become ETH for the LP” — show in Preview
3. Same-chain, native coin	0.5 ETH on Base → ETH/USDC pool	(1) keep ETH as native (V4) or wrap to WETH (V3) · (2) split-swap half · (3) mint	V4 native path skips wrap; V3 must wrap
4. Cross-chain, same token	\$2k USDC on Ethereum → USDC/SOL on Raydium CLMM (Solana)	(1) approve USDC on ETH · (2) deBridge DLN to Solana · (3) split half to SOL via Jupiter · (4) Whirlpool/Raydium CLMM open_position	Step 3 + 4 fire after pending_dst_fill clears (§6c)
5. Cross-chain, different token	\$2k USDT on Ethereum → JLP on Solana	(1) approve USDT · (2) deBridge to USDC(SOL) · (3) Jupiter Perps LP deposit	USDT not directly bridge-able to JLP-eligible token; route via USDC
6. Cross-chain, native source	0.6 ETH on Arbitrum → ETH/USDC on Base	(1) deBridge DLN ETH(ARB) → ETH(Base) · (2) split-swap half on Base · (3) mint	Solver fills with native or WETH depending on route
7. Dust mixing	Multiple small balances totaling \$1.5k on Polygon	(1) Enso shortcut: sweep all → USDC · (2) split-swap · (3) deposit	Enso's multi-input bundling shines here; flag if any input is below \$10 (gas not worth it)
8. Partial allowance	User already approved 1000 USDC; needs 2000 for new deposit	(1) increase allowance (or Permit2 sig for delta) · (2) mint	Builder always checks current allowance; never re-approves wastefully
9. Gas token missing on destination chain	User wants to LP on Sonic but has no S for gas	(1) deBridge a small ETH→S (gas-only) ahead of main flow · (2) main bridge · (3) mint	Auto-detect; offer to bundle. Min gas top-up: 1.5× estimated gas cost

# Scenario	Concrete example	Ordered plan	Notes
10. Pre-deposited LST	User holds wstETH, wants ETH/USDC LP	(1) unwrap wstETH → stETH · (2) swap stETH → ETH (Curve stETH/ETH) · (3) split + mint	Or alternatively offer wstETH/WETH LP if available (e.g., Curve, Balancer)
11. NFT-locked LP refinance (rebalance)	User has Uniswap V3 ETH/USDC NFT, ETH moved out of range	(1) decreaseLiquidity 100% · (2) collect fees · (3) burn old NFT · (4) mint new NFT with updated range	Bundle in one tx via <code>NFPM.multicall</code> for V3/V4; on Solana <code>close_position</code> + <code>open_position</code>
12. Claim-and-compound	User has Slipstream position, AERO emissions accrued	(1) <code>gauge.getReward</code> · (2) swap AERO → token0/token1 in optimal ratio · (3) <code>increaseLiquidity</code>	Show expected position growth in Preview; AERO often more valuable held than zapped
13. Single-sided into single-sided protocol	User has 1 ETH, wants Aave V3 supply on Base	(1) approve · (2) supply	Simplest case; aToken receipt; no range; eMode auto-detected for ETH correlated assets
14. Migrate position to new pool	User has Uniswap V2 ETH/USDC LP, wants to migrate to V3 0.05%	(1) V2 <code>removeLiquidity</code> · (2) approve both tokens · (3) V3 mint	Bundle in one tx via migrator contract or multicall
15. Wrong wallet for chain	User has Solana wallet only, intent is on EVM (or vice versa)	Block at intent stage; show wallet-connect prompt for the right chain family	Phantom can sign EVM if enabled; Solflare/raw EVM wallets cannot sign Solana

8. Protocol-Family Matrix

Master reference table. One row per protocol family × pool type. Use this as the index into Chapter 9 deep dives.

Family	Chains	Pool type	Range support	Single-sided	SDK / API	Receipt	Notable aggregator gap	Deposit shape
Uniswap V3	EVM (15 chains)	CLMM	Yes (tick)	Yes (one-sided)	@uniswap/v3-sdk	ERC-721 NFT (NFPM)	—	NFPM.mint via multicall
Uniswap V4	EVM (~7 chains as of 2026-05)	CLMM + hooks	Yes (tick)	Yes	@uniswap/v4-sdk	ERC-721 (PositionManager)	Hooks not in 1inch/Enso routes	PositionManager.modifyLiquidity([MINT, SETTLE, TAKE])
PancakeSwap V3	BSC, ETH, Base, Arb, zkSync, Linea, Polygon zk	CLMM	Yes (tick)	Yes	@pancakeswap/v3-sdk + masterChef-V3	ERC-721 NFT	—	PancakeNFPM.mint + masterChef.stake
Aerodrome Slipstream	Base	CLMM (tickSpacing-keyed)	Yes (tick)	Yes	Custom resolver + UniV3 SDK shape	ERC-721 NFT + optional CLGauge	Slipstream pools not always in Enso	NFPM.mint + (opt) CLGauge.deposit(tokenId)
Velodrome CL	Optimism, Mode	CLMM (tickSpacing-keyed)	Yes (tick)	Yes	Same as Slipstream	ERC-721 NFT + CLGauge	—	Same as Slipstream
Uniswap V2 + forks (Sushi V2, PancakeSwap V2, Quickswap V2, Camelot V2, Velodrome V1, Aerodrome V1, Baseswap, Trader Joe V1, Spookyswap, Spiritswap)	Per fork (every chain)	AMM v2	No (full-range only)	No (Enso zap-in fakes it)	Router02 ABI + Enso zap	ERC-20 LP	—	Router.addLiquidity / addLiquidityETH
Curve (legacy plain/lending/crypto)	EVM (12 chains)	Stable / Crypto AMM	No	Yes (single-coin add)	curve-js + per-pool ABI	ERC-20 LP + gauge	Some factory pools missing from aggregators	add_liquidity(uint256[N], min_mint_amount)
Curve stable-ng / twocrypto-ng	EVM (newer chains, Berachain, Sonic)	Stable / Crypto AMM, dynamic-len	No	Yes	curve-js metaregistry	ERC-20 LP	ABI differs from legacy	add_liquidity(uint256[], min_mint, receiver)
Balancer V2	EVM (8 chains)	Weighted / ComposableStable / Boosted	No (weighted has ratios)	Yes (joinKind)	@balancer-labs/sdk	BPT (ERC-20)	—	Vault.joinPool(poolId, sender, recip, JoinPoolRequest)
Balancer V3	ETH, Arb, Gnosis (more rolling)	Custom AMM logic via hooks; weighted/stable rewritten	No (depends on pool)	Yes	@balancer/sdk V3	BPT (ERC-20)	Permit2 mandatory	Router.addLiquidity({Proportional, Unbalanced, SingleTokenExactOut})
Aave V3	EVM (all 17)	Single-sided lending pool	N/A	Yes (only sided)	@aave/contract-helpers	aToken (ERC-20)	—	Pool.supply(asset, amount, onBehalfOf, refCode)

Family	Chains	Pool type	Range support	Single-sided	SDK / API	Receipt	Notable aggregator gap	Deposit shape
Compound V3 (Comet)	ETH, Arb, Base, Polygon, Optimism, Mantle	Single-asset supply (base) + multi-collateral	N/A	Yes	Comet ABI direct	cBaseToken (ERC-20)	—	Comet.supply(asset, amount)
Morpho Blue / MetaMorpho	ETH, Base, Arb, Polygon	Isolated lending markets / vault aggregator	N/A	Yes	Morpho SDK	MetaMorpho share (ERC-4626) or Morpho-Blue position	—	Morpho.supply / MetaMorpho.deposit
Spark / Sky (sDAI / sUSDS)	ETH, Base, Gnosis	Wrapped stablecoin yield	N/A	Yes	Direct sToken contract	ERC-4626 share	—	<i>deposit/assets, receiver</i>
Pendle V2	ETH, Arb, BSC, Optimism, Mantle	Yield-tokenization (PT + YT + LP)	Partial (LP has impl. range)	Yes (mintPY only / PT-only zap)	Pendle SDK	PT + YT + LP ERC-20s	Some routes missing	Router.mintPyFromToken / addLiquidity
Stargate V2	EVM (10 chains)	Single-token bridge LP	N/A	Yes	Direct router ABI	S*USDC/S*USD (ERC-4626-like)	—	<i>deposit(token, amount, to)</i>
ERC-4626 vaults (Yearn V3, Beefy, Reaper, Idle, Sommelier, Origin, Etherfi WeETHs, Renzo ezETH)	EVM (varies)	Vault wrappers (over CL, lending, LST)	N/A (underlying may have)	Yes	ERC-4626 interface	Vault share (ERC-20)	—	<i>deposit/assets, receiver</i>
GMX V2 (GM markets) / GLP	Arb, Avalanche	Synthetic perp LP	No	Yes (deposit single)	GMX V2 SDK	GM token (ERC-20) or GLP	—	Router.createDeposit (GM); GlpManager.addLiquidity (GLP legacy)
Lido (stETH / wstETH)	ETH, Polygon	LST	N/A	Yes	Direct submit() or curve fallback	stETH / wstETH (ERC-20)	—	Lido.submit(referral) with msg.value
Rocket Pool (rETH)	ETH	LST	N/A	Yes (when not capped)	Direct deposit pool or secondary	rETH (ERC-20)	Capped on direct mint	DepositPool.deposit
Frax (sfrxETH / frxETH)	ETH, Arb	LST + autocompounding wrapper	N/A	Yes	Direct frxETHMinter	frxETH / sfrxETH	—	frxETHMinter.submit
Stader ETHx, Mantle eETH, Etherfi weETH	ETH (mETH also Mantle)	LST	N/A	Yes	Direct deposit contract	Token (ERC-20)	—	<i>deposit()</i> payable
Renzo ezETH, Kelp rsETH, Swell rswETH, Puffer pufETH, EigenLayer LRT restake	ETH (some Arb, Base)	LRT	N/A	Yes	Direct deposit contract	Token (ERC-20)	Often not Enso-routable	<i>deposit()</i> or Zapper variant

Family	Chains	Pool type	Range support	Single-sided	SDK / API	Receipt	Notable aggregator gap	Deposit shape
Raydium AMM v4	Solana	AMM v2-style	No	Yes (Jupiter zap)	raydium-sdk-v2	LP mint (SPL)	LP mint not Jupiter-swappable	raydium.liquidity.addLiquidity
Raydium CPMM	Solana	Constant-product (new-gen)	No	Yes	raydium-sdk-v2	LP mint (SPL)	—	raydium.cpmm.addLiquidity
Raydium CLMM	Solana	CLMM	Yes (tick)	Yes	raydium-sdk-v2	Position-NFT (SPL)	—	raydium.clmm.openPositionFromBase
Orca v1 AMM (legacy)	Solana	AMM v2-style	No	Yes (Jupiter zap)	Deprecated — delegate to Jupiter	LP mint	Most pools migrated	Legacy
Orca Whirlpools	Solana	CLMM	Yes (tick)	Yes	@orca-so/whirlpools	Position-NFT (Metaplex)	—	open_position_with_metadata + increase_liquidity
Meteora DLMM	Solana	Bin-based DLMM	Yes (bin)	Yes (BidAsk strategy)	@meteora-ag/dlmm	Position-PDA (no NFT)	Bin-arrays must be initialized	initializePositionAndAddLiquidityByStrategy
Meteora DAMM v2 + Dynamic Vaults	Solana	AMM with dynamic fees / vault wrappers	No / N/A	Yes	@meteora-ag/* SDKs	LP mint / vault share	Token-2022 support varies	Per-product addLiquidity
Kamino Vaults V1	Solana	Auto-managed CLMM wrapper	N/A (vault picks range)	Yes	Kamino SDK	Strategy share SPL	—	kvault.deposit(strategy, amount)
Kamino Lend (klend)	Solana	Lending market	N/A	Yes	Kamino SDK	Obligation state	—	klend.deposit_reserve_liquidity_and_obligation_collateral
JLP (Jupiter Perps LP)	Solana	Perp counterparty LP	N/A (basket)	Yes (single token in)	Jupiter Perps program direct	JLP (SPL)	—	perp_lp.add_liquidity(amount_in, min_lp_out)
Marinade native + mSOL	Solana	LST	N/A	Yes	Marinade SDK	mSOL (SPL) or native stake account	—	marinade.deposit or depositStakeAccount
JitoSOL	Solana	LST	N/A	Yes	Jito Stake Pool program direct	JitoSOL (SPL)	—	stake_pool.deposit_sol
Sanctum INF / Infinity-pools	Solana	LST router / basket	N/A	Yes	Sanctum SDK	INF token (SPL)	—	sanctum.add_liquidity

9. Per-Family Deep Dives

Weighted depth: full pages for the 13 families that account for the bulk of LP TVL on Ilyon's chains. One-paragraph stubs in Chapter 10 for V2 forks, long-tail Solana, and small CL forks.

9a. Uniswap V3

Pool model. Concentrated liquidity in tick ranges. Each pool is a unique contract deployed by the V3 Factory, keyed by $(token0, token1, fee_bps)$ with fee tiers 100 / 500 / 3000 / 10000 (1 / 5 / 30 / 100 bp). $tickSpacing$ is derived: 1 / 10 / 60 / 200.

Receipt. ERC-721 NFT minted by the NonfungiblePositionManager (NFTM). Position state (liquidity, tickLower, tickUpper, fee growth checkpoints, owed tokens) lives in the NFTM indexed by tokenId.

Deposit shape — conceptual call sequence:

```
1. approve token0 to NFTM (or Permit2 sig)
2. approve token1 to NFTM
3. NFTM.multicall([
  NFTM.mint({ token0, token1, fee, tickLower, tickUpper,
    amount0Desired, amount1Desired,
    amount0Min, amount1Min, recipient, deadline }),
  NFTM.refundETH() // if msg.value was sent
])
```

Range semantics. Lower and upper are *ticks* (integer). Convert from price via $tick = \text{floor}(\log_{1.0001}(\text{price}))$, then round to nearest valid tick ($tick - tick \% tickSpacing$). The current pool tick determines the deposit ratio: if $tick < tickLower$, deposit is 100% token0; if $tick > tickUpper$, 100% token1; in between, ratio is determined by L (LiquidityAmounts.getAmountsForLiquidity).

Single-sided support. Yes — by placing the range entirely above or below the current tick. Useful for limit-order-like LP: lower bound > current price, deposit 100% token1, earn fees as price rises through the range and converts to token0.

Fee tiers in practice. Stable/stable → 100/500 (1 / 5 bp); blue-chip/stable → 500 (5 bp); blue-chip/blue-chip → 500 or 3000; exotic/blue-chip → 3000 or 10000.

Known unsupported aggregator routes. 1inch and 0x are swap-only, not position-aware. Enso has good V3 coverage on Ethereum/Base/Arbitrum/Optimism/Polygon but spotty on Linea, Scroll, Mantle, Blast. KyberSwap doesn't do position mints. For position mints always use the V3 SDK; use aggregators only for the pre-swap leg.

Recommended SDK / router. @uniswap/v3-sdk for math (TickMath, LiquidityAmounts, SqrtPriceMath). Permit2 from @uniswap/permit2-sdk. Pre-swap leg via Enso shortcuts (EVM catch-all, position-aware) OR Uniswap's Universal Router (integrated) OR 1inch v6 for best price (but adds an approval to 1inch router).

Minimum data from user. Pair, fee tier (or auto-default), one of {amount0, amount1, totalUSD}, range (preset or custom). Everything else derived.

9b. Uniswap V4

Pool model. Singleton PoolManager holds all pools' state. Pools are identified by $poolId = \text{keccak256}(\text{abi.encode}(\text{PoolKey}))$ with $\text{PoolKey} = (\text{currency0}, \text{currency1}, \text{fee}, \text{tickSpacing}, \text{hooks})$. No per-pool contracts; storage is indexed by poolId. Hooks introduce arbitrary callback logic at lifecycle events (beforeSwap, afterSwap, beforeAddLiquidity, afterAddLiquidity, etc.).

Receipt. ERC-721 NFT from the V4 PositionManager (separate periphery contract). Position state lives in PoolManager, indexed by $(poolId, owner, salt)$.

Deposit shape — uses the unlock-callback pattern:

```
1. Permit2 sig for token0 and token1 (V4 requires Permit2)
2. PositionManager.modifyLiquidities(
  abi.encode(actions, params), // actions = [MINT_POSITION, SETTLE_PAIR, TAKE_PAIR]
  deadline
)
The PositionManager calls PoolManager.unlock(callbackData);
inside unlockCallback, mint adds liquidity then SETTLE_PAIR pays the owed delta
and TAKE_PAIR collects any positive delta. All in one tx.
```

Range semantics. Same tick math as V3. tickSpacing is part of PoolKey, so the same (currency0, currency1, fee) can have multiple pools with different tickSpacings **and different hooks**. The resolver MUST surface this in chat — “3 V4 pools match ETH/USDC: 5bp/1-tick/no-hooks, 5bp/10-tick/dynamic-fee-hook-X, 30bp/60-tick/no-hooks.”

Hook handling. Hooks are arbitrary code. Shield must (a) maintain an allowlist of audited hook addresses, (b) refuse unknown hooks unless user explicitly opts into experimental pools, (c) re-scan hook contracts on every plan (hook can be upgradeable). Some hooks change behavior: dynamic fees, MEV-skim, LVR-mitigation, KYC gates. Each is a Preview-card warning.

Native ETH path. V4 supports currency0 = *address(0)* (native). Builder must NOT wrap to WETH for native-V4 pools; just pass msg.value. This is the cleanest LP UX available on EVM for ETH/stable pairs.

v3 vs v4 side-by-side diff.

Aspect	V3	V4
Pool identity	Contract address	poolId = keccak(PoolKey)
Storage	Per-pool contract	Singleton PoolManager
Approvals	Approve NFPM directly or Permit2	Permit2 mandatory
Native ETH	Wrap to WETH	Native supported via currency0 = address(0)
Mint entrypoint	NFPM.mint	PositionManager.modifyLiquidities([MINT,SETTLE,TAKE])
Receipt	ERC-721 from NFPM	ERC-721 from V4 PositionManager
Range params	(tickLower, tickUpper, fee)	(tickLower, tickUpper, salt)
Hooks	None	Five lifecycle hooks per pool
Fee model	Static per tier	Static OR dynamic (hook-driven)
Aggregator coverage	Mature	Sparse; native SDK preferred

Recommended SDK / router. @uniswap/v4-sdk (Actions, V4Planner, PoolKey helpers). Pre-swap leg via Uniswap Universal Router (now V4-aware) or Enso. Hook registry: pull from Uniswap’s curated V4 hook list + Ilyon’s allowlist.

Versioning posture. V4 ships behind a feature flag on a per-chain basis. V3 remains default until V4 TVL on a chain > 20% of total V3+V4 TVL for that chain or a flagship user-favored pool migrates.

9c. PancakeSwap V3

Pool model. Direct Uniswap V3 fork with the same NFPM ABI shape. Fee tiers are 100 / 500 / 2500 / 10000 (1 / 5 / 25 / 100 bp — note 25 bp not 30 bp). tickSpacing is 1 / 10 / 50 / 200.

Distinguishing feature: CAKE emissions via MasterChef V3. Positions staked into MasterChef V3 earn CAKE rewards in addition to swap fees. Unlike Slipstream/Velodrome, fees are **retained by the LP** while staked — no forfeit. This makes “stake-into-MasterChef” the default recommendation when CAKE emissions are non-zero.

Deposit + stake sequence:

```
1. approve token0, token1 (or Permit2)
2. PancakeNFPM.mint({ ..., recipient: userAddress })
3. PancakeNFPM.safeTransferFrom(user, masterChefV3, tokenId)
■ MasterChef's onERC721Received auto-stakes the position
(Steps 2 + 3 bundled via multicall when supported)
```

Chains. BSC (primary), Ethereum, Base, Arbitrum, zkSync Era, Linea, Polygon zkEVM. Coverage per pool varies — always check via the on-chain factory before building.

Recommended SDK / router. @pancakeswap/v3-sdk (compatible with v3-sdk shape) + MasterChef V3 ABI direct. Pre-swap via 1inch v6 or KyberSwap (both have strong BSC coverage); Enso also works on Ethereum/Base/Arbitrum.

9d. Aerodrome Slipstream & Velodrome CL

Pool model. Forked from Uniswap V3 (periphery 6cce88e). The factory is keyed by (*token0*, *token1*, *tickSpacing*), not (*token0*, *token1*, *fee*). Multiple tickSpacings share NFPM. Common tickSpacings: 1 (≈1 bp), 50 (≈5 bp), 100 (≈10 bp), 200 (≈30 bp), 2000 (≈1%).

The tickSpacing-vs-fee gotcha.

- `factory.getPool(t0, t1, tickSpacing) → poolAddress` — NOT (... , fee).
- The pool itself has a `fee()` getter but it's set per-tickSpacing at deployment.
- Resolver MUST enumerate tickSpacings, never fees. Mapping back to fee for UI display is OK but the canonical key in the plan is tickSpacing.

Distinguishing feature: CLGauge fee-vs-emission trade-off. Each CL pool has an associated CLGauge contract. Depositing the position-NFT into the gauge earns AERO (or VELO) emissions, but **the LP forfeits swap fees to veAERO/veVELO voters**. This is the opposite of PancakeSwap V3's MasterChef. Surface this trade-off as a Preview toggle, OFF by default, with both APR streams quoted separately.

Deposit sequence:

```
1. approve token0, token1 to NFPM
2. NFPM.mint({ token0, token1, tickSpacing, tickLower, tickUpper, ... })
3. (optional) NFPM.safeTransferFrom(user, CLGauge, tokenId)
■ stakes for AERO/VELO emissions, forfeits fees
```

Recommended SDK / router. No vendor SDK exists; use a custom resolver that wraps the V3 SDK math with Slipstream/Velodrome NFPM ABIs. Pre-swap on Base → Aerodrome's own V2 router for AERO-related pairs; otherwise Enso or 1inch v6.

APR card. Must show three numbers: (1) swap-fee APR (if not staked), (2) AERO/VELO emission APR (if staked), (3) total. Most users want (2) for active gauge pools — emissions often dominate fees on Base/Optimism.

9e. Curve

Pool model. Three major families, each with a different ABI:

- **Legacy plain pools** (stETH/ETH, 3pool, sUSD): `add_liquidity(uint256[N] amounts, uint256 min_mint)` — fixed-size array.
- **Crypto pools** (tricrypto, crvUSD): same shape but with internal price oracles and different fee model.
- **Stable-NG / Twocrypto-NG** (newer factory pools, default on newer chains like Berachain, Sonic): `add_liquidity(uint256[] amounts, uint256 min_mint, address receiver)` — dynamic-size array, recipient param. **This is the breaking ABI change.**

Receipt. ERC-20 LP token. Optional staking into *LiquidityGauge* for CRV emissions; deposit-time gauge boost requires veCRV balance.

Range semantics. None — Curve is full-range stable/crypto AMM. APR is purely fee + reward, no in-range probability question.

Single-sided support. Yes — `add_liquidity` accepts any subset of coin amounts (others zero); the pool charges an imbalance fee proportional to deviation from optimal ratio. `calc_token_amount(amounts, true)` previews the LP out (but excludes fees — always warn the user that the actual receive is slightly lower).

Versioning side-by-side:

Aspect	Legacy	Stable-NG / Twocrypto-NG
add_liquidity sig	(uint256[N], uint256)	(uint256[], uint256, address)
Receiver param	Always msg.sender	Explicit recipient (enables zap routers)
LP token	Separate token contract	LP is the pool itself (single contract)
Factory	Per-pool-type	Unified factory
Gauge	Separate gauge	Same — separate
Default on	Older chains, mainnet legacy pools	Newer chains, all new pools 2024+

Recommended SDK / router. *curve-js* + the on-chain *metaregistry* (returns pool params, coins, balances, n_coins, is_meta, is_ng). Pre-swap via Curve's own *exchange* router (best for stable→stable inside the Curve ecosystem) or Odos (best Curve-aware aggregator).

9f. Balancer V2 + V3

V2 pool model. Multi-token pools with arbitrary weights (Weighted), pegged-asset AMM (ComposableStable), or rebase-yield aggregator (Boosted). All pools share the singleton Vault. Each pool has a *poolId* (bytes32) embedding pool address + type.

V2 deposit shape.

```
Vault.joinPool(
  poolId,
  sender,
  recipient,
  JoinPoolRequest({
    assets: [token0, token1, ..., tokenN, BPT_for_composable_stable],
    maxAmountsIn: [...],
    userData: abi.encode(joinKind, amountsIn, minBPTOut),
    fromInternalBalance: false
  })
)
```



```
joinKind ∈ { EXACT_TOKENS_IN_FOR_BPT_OUT, TOKEN_IN_FOR_EXACT_BPT_OUT,
ALL_TOKENS_IN_FOR_EXACT_BPT_OUT,
INIT (first-time only for ComposableStable) }
```

V3 — the rewrite. Balancer V3 keeps the singleton Vault concept but rebuilt internals: transient storage for atomic settlement (mirroring Uniswap V4), custom AMM logic via hooks, mandatory Permit2 for approvals, new Router contract (separate from Vault). The user-facing endpoints are *Router.addLiquidityProportional* / *addLiquidityUnbalanced* / *addLiquiditySingleTokenExactOut* / *addLiquidityCustom*.

V2 vs V3 side-by-side:

Aspect	V2	V3
Entrypoint	Vault.joinPool	Router.addLiquidity{Proportional,Unbalanced,SingleTokenExactOut,Custom}
Approvals	Approve Vault (or Permit2)	Permit2 mandatory
BPT location	Per-pool ERC-20 contract	Per-pool ERC-20, but custody by Vault
Pool logic	Pool contract implements join math	Vault implements; pool provides hook funcs only
Hooks	None	Pre/post-swap, pre/post-liquidity
Pool types live	Weighted, ComposableStable, Boosted	Weighted, Stable, Gyro (custom hooks)
Chains	8+ (mature)	ETH, Arb, Gnosis (rolling)

Receipt. BPT (ERC-20). Optional staking into Balancer gauges for BAL emissions; veBAL boost similar to Curve.

Recommended SDK / router. *@balancer/sdk* (covers both V2 and V3). Pre-swap via Balancer's own SOR (Smart Order Router) for in-Balancer routes; Enso or 1inch for cross-protocol pre-swaps. **Always use BalancerHelpers.queryJoin** in V2 (or *Router.queryAddLiquidity** in V3) for accurate BPT-out estimation — never compute off-chain.

9g. Aave V3 (single-sided) and Pendle V2 (yield-tokenization)

Aave V3

Pool model. Money market with per-asset reserves. Supply is single-sided. No range. eMode allows higher LTV for correlated asset baskets (ETH-correlated, stable-correlated, MATIC-correlated). Isolation mode restricts new listings to single-collateral debt with a debt ceiling.

Receipt. aToken (ERC-20, rebasing). 1:1 with supplied asset by exchange-rate. Interest accrues as the aToken balance grows.

Deposit shape:

1. approve asset to Pool
2. Pool.**supply**(asset, amount, onBehalfOf, referralCode)
(or supplyWithPermit when EIP-2612 supported)

Edge cases: some markets are *frozen* (paused for new supply) — check *getReserveConfiguration*. Isolated-mode assets cannot be supplied alongside other collateral on the same user account. New listings often have *supply caps* — Builder must check *getReserveCaps* and refuse with “market at supply cap”.

Recommended SDK. @aave/contract-helpers + the on-chain *UiPoolDataProviderV3* for reserve state. No pre-swap router needed for supply; for the source-token-to-supply-asset leg, use Enso or 1inch.

Pendle V2

Pool model. Yield tokenization: each market has an SY (Standardized Yield) wrapper of an underlying yield-bearing asset (e.g., stETH, sDAI, weETH), and the SY can be split into PT (Principal Token, redeems for 1 unit of underlying at maturity) + YT (Yield Token, accrues the yield until maturity). Markets also have an AMM-like PT/SY pool for LP.

Three deposit entry points, each a distinct user journey:

- **mintPyFromToken:** split underlying into PT + YT. Used for yield-trading.
- **swapTokenForPt:** zap straight into PT for fixed-rate exposure (implicit yield discount).
- **addLiquidityFromToken:** deposit into PT/SY LP — earns swap fees + base yield from SY.

Range semantics. Pendle LP has implicit range tied to PT price (between 0 and 1 underlying-equivalent). No explicit range parameter, but as maturity approaches, the AMM behavior changes — the LP becomes less efficient near expiry. Surface the maturity date in Preview.

Async behavior (epoch boundaries). Some Pendle markets have epoch-based entry (weekly reward checkpoints). The Builder must check *market.getReserveData* for the next epoch boundary; if entry within 5 minutes of an epoch, queue with a *pending_epoch_entry* blocker and execute at the boundary.

Receipt. PT + YT (ERC-20s) for mintPY; PT (ERC-20) for PT-only zap; LP (ERC-20) for addLiquidity.

Recommended SDK. @pendle/sdk-v2 + *Pendle Router*. Coverage of SY wrappers is the bottleneck — new yield-bearing assets often lag a few weeks before Pendle deploys an SY. Maintain a Pendle-supported-asset registry.

9h. Raydium AMM v4 / CPMM / CLMM

Three distinct Raydium pool families with different mechanics, deposit shapes, and receipts. The resolver must always disambiguate.

Raydium AMM v4 (legacy)

Constant-product AMM with an OpenBook market integration (some pools), full-range. Receipt: LP mint (SPL token). **Critical gotcha: LP mints are not Jupiter-tradable** — redemption must go through Raydium's own program. Builder must store the LP-redeem path in the position metadata.

Raydium CPMM (constant-product, new-gen)

Cleaner constant-product without the OpenBook dependency. Receipt: LP mint (SPL). Jupiter-tradable in most cases. Lower per-tx cost. **Default Raydium AMM-style choice for 2026.**

Raydium CLMM

Concentrated liquidity (Uniswap V3-style) on Solana. Receipt: position-NFT (SPL with Metaplex metadata) + position-PDA. Has its own reward emission system distinct from Solana DEX fees.

CLMM deposit conceptual sequence:

```
1. pre-flight: ensure both token ATAs exist (create_idempotent if missing)
ensure WSOL wrapped if SOL-side
compute (tickLower, tickUpper) rounded to pool.tickSpacing
ensure TickArray accounts exist for the range (initialize if not)
2. raydium.clmm.openPositionFromBase({
  pool, tickLowerIndex, tickUpperIndex,
  baseAmount, baseSide: 0|1, // which token is the user-specified side
  otherAmountMax, // slippage cap on the other side
  priorityFee
})
■ atomically: mints position-NFT, creates position-PDA, deposits liquidity
```

Recommended SDK. [@raydium-io/raydium-sdk-v2](https://github.com/raydium-io/raydium-sdk). The SDK exposes a fluent TransactionBuilder that handles ATAs, WSOL wrap, tickArray init, ALT lookup, and priority fees. Pre-swap via Jupiter Swap API or Jupiter Ultra (Ultra has RFQ + MEV protection but higher fee surface; default to Swap for now).

9i. Orca Whirlpools

Pool model. Concentrated liquidity with pool fee tiers identified by *fee_tier_index* (not *fee_bps* directly). Common *fee_tier_index* values map to tickSpacings of 1, 8, 64, 96, 128, 256. Recent versions introduced **fee_tier_index** as the canonical key (replacing the older *direct-feeRate* field).

Receipt. Position-NFT (Metaplex metadata token) plus a position-PDA holding state. **open_position_with_metadata** mints a metadata token (better wallet display); **open_position** mints a bare position mint.

Deposit conceptual sequence:

```
1. pre-flight: ATAs + WSOL wrap + sync_native + tickArray init
2. whirlpool.open_position_with_metadata(
  positionAuthority, fundingWallet,
  whirlpool, tickLowerIndex, tickUpperIndex,
  positionMint, positionMetadataAccount // PDA from positionMint
)
3. whirlpool.increase_liquidity(
  liquidityAmount, tokenMaxA, tokenMaxB, ... // amounts cap slippage
)
```

Splash Pools. A 2024+ feature: zero-config CL pools auto-deployed for memecoin launches with a single tickSpacing preset. Whirlpool SDK exposes *create_splash_pool* and a simpler one-tap deposit. For Splash Pools, the chat skips range selection entirely (only Full or “Auto” offered).

Token-2022 support. Whirlpools has rolled out Token-2022 support per-pool. Builder must check pool’s *token_program* and use *open_position_2022* / *increase_liquidity_2022* variants. Transfer hooks are honored — if the token has a hook program, that program must be invoked during deposits (the SDK handles this).

Recommended SDK. *@orca-so/whirlpools* (the Rust-derived TypeScript SDK; the older *@orca-so/sdk* is deprecated). Pre-swap via Jupiter Swap API.

Minimum data from user. Pair, one of {amount0, amount1, totalUSD}, range (preset). Everything else derived. *fee_tier_index* resolved by the Pool Index from TVL ranking.

9j. Meteora DLMM / DAMM v2 / Dynamic Vaults

DLMM

Pool model. Discrete liquidity bins. Each bin is a single price; swaps inside a bin have **zero slippage**. Bin step (in bps) defines the price ratio between adjacent bins: 1 bp, 5 bp, 20 bp, 100 bp typical. Active bin (*active_id*) is the current trading bin.

Receipt. Position-PDA (NOT an NFT — important distinction from Whirlpool / Raydium CLMM). The position is owned by the user’s wallet directly via PDA seeds. To transfer, the user reassigns the PDA owner — rare.

Strategy types (liquidity distribution across bins):

- **Spot:** uniform across bins in range. Maximum fee capture if price stays in range.
- **Curve:** bell-shaped, concentrated at *active_id*. Better in-range probability near current price.
- **BidAsk:** U-shaped, concentrated at edges. Functions as a range-order / limit-order (deposit one side at edge, fills as price reaches it).

Deposit conceptual sequence:

```
1. pre-flight: ATAs + WSOL wrap + binArray init (each binArray covers 70 bins)
2. dlmmPool.initializePositionAndAddLiquidityByStrategy({
  positionPubKey, user, totalXAmount, totalYAmount,
```

```
strategy: { minBinId, maxBinId, strategyType: Spot|Curve|BidAsk }  
})  
■ atomically: creates position-PDA + deposits according to strategy
```

autoFillByStrategy. If user gave only X amount, the SDK helper computes the optimal Y for the chosen strategy and range. Builder always uses this rather than computing manually.

Token-2022 + transfer hooks. Newer DLMM pools support Token-2022; legacy ones don't. Builder must check pool config and route appropriately.

DAMM v2 + Dynamic Vaults

DAMM v2 is Meteora's evolution of the dynamic AMM (originally Mercurial-derived). Dynamic fees adjust to volatility. Receipt: LP mint. Full-range. Use Meteora's SDK; Jupiter-tradable in most cases.

Dynamic Vaults are yield-routing wrappers that allocate idle liquidity across lending markets (Save, Marginfi, etc.) and farms. Receipt: vault share SPL. Single-asset deposits only.

Recommended SDKs. [@meteora-ag/dlmm](#), [@meteora-ag/dynamic-amm-sdk](#), [@meteora-ag/vault-sdk](#).
Pre-swap via Jupiter.

9k. Kamino Vaults V1 + Multiply Vaults + Kamino Lend (klend)

Vaults V1

Pool model. Auto-managed CL vault wrappers over Whirlpool, Raydium CLMM, and Meteora DLMM. The vault picks the range, rebalances on its own schedule, and reinvests fees. From the user's perspective, a Kamino vault is a single-deposit yield product — no range to pick.

Receipt. Strategy-share SPL token (kToken-style). Redeem via *kvault.withdraw*.

Deposit conceptual sequence:

```
1. pre-flight: ATAs + WSOL wrap
2. kvault.deposit({ strategyPubkey, tokenAAmount, tokenBAmount })
■ vault converts to its internal ratio, mints kToken shares to user
```

Multiply Vaults. Looped-leverage products (e.g., JLP looped through klend for amplified yield). High blow-up risk. Surface explicitly in Preview with a “Leverage X×” warning and the LTV liquidation distance.

Klend (Kamino Lend)

Lending market with isolated and cross-margin modes. Deposits go into a reserve and credit the user's obligation account.

Deposit shape: *deposit_reserve_liquidity_and_obligation_collateral(reserve, obligation, amount)*. The obligation must be initialized first if it doesn't exist.

Receipt. Reserve collateral state in the obligation account (no separate SPL). Verification = `obligation.deposits[reserve].amount delta`.

Recommended SDK. *@kamino-finance/klend-sdk* for klend, and *@kamino-finance/kamino-sdk* for vaults. Pre-swap via Jupiter.

9l. JLP / JitoSOL / Marinade / Sanctum INF

JLP (Jupiter Perps LP)

Pool model. Single-token-in, basket-out. The LP is the counterparty for Jupiter Perpetuals; the LP basket is SOL + ETH + BTC + USDC + USDT in dynamic weights. Deposit a single token; receive JLP minted at the basket NAV.

Receipt. JLP SPL token. Yield comes from perp fees + position PnL (LP is counterparty).

Deposit shape:

```
1. pre-flight: ATAs + WSOL wrap if SOL-side
2. jupiter_perps.add_liquidity({
  token_in, // any of SOL/ETH/BTC/USDC/USDT
  amount_in,
  min_jlp_out // slippage cap, given current NAV
})
```

Withdrawal lockup. Recent versions of Jupiter Perps impose a withdrawal cool-down (usually 1 hour). Surface this in Preview — “your funds are locked for 1 hour after deposit”.

JitoSOL

Liquid staking pool managed by Jito. Deposit SOL → receive JitoSOL (SPL); JitoSOL accrues stake rewards + MEV revenue. Single-token-in, no range. Receipt: JitoSOL SPL. Use *SPL Stake Pool program.deposit_sol* instruction.

Marinade

Two paths: (a) *deposit* SOL → mSOL (instant, liquid); (b) *depositStakeAccount* a pre-existing stake account (no cool-down for staked SOL). Marinade also runs **native staking** (no LST, but no liquidity).

Async behavior. mSOL minting is instant; underlying stake activation is next-epoch. For users who don't need liquid LST, native-stake is yield-superior. Surface the trade-off in Preview.

Sanctum INF

Multi-LST router: deposit any supported LST (JitoSOL, mSOL, bSOL, etc.) into the Infinity Pool; receive INF (SPL). INF is the LP for swapping between LSTs at minimal slippage. APR = swap fees from LST-routing + underlying LST yields.

Recommended SDKs. Direct program access for Jupiter Perps and Jito Stake Pool; Marinade has [@marinade.finance/marinade-ts-sdk](https://github.com/marinade-finance/marinade-ts-sdk); Sanctum has its own SDK. Pre-swap via Jupiter.

9m. LST / LRT Hub Set

The LST/LRT landscape has 15+ receipt-bearing wrappers, many of which are **not indexed by EVM aggregators** (Enso, 1inch). For each, the mint path is either a direct *deposit()* on the protocol contract or a secondary-market swap (Curve, Balancer, Uniswap).

Protocol	Token(s)	Chains	Mint path	Aggregator-indexed?	Notes
Lido	stETH / wstETH	ETH, Polygon	stETH. <i>submit(ref)</i> payable; wstETH = wrap stETH	Yes — both Enso and Curve route	Mature; capped during congestion
Rocket Pool	rETH	ETH	DepositPool. <i>deposit()</i> payable	Direct mint often capped — use Balancer rETH/ETH	Secondary market is the reliable path
Frax	frxETH / sfrxETH	ETH, Arb	frxETHMinter. <i>submit()</i> + sfrxETH. <i>deposit</i>	Yes via Curve frxETH/ETH	sfrxETH is autocompounded ERC-4626 wrap
Stader	ETHx	ETH	StakePoolManager. <i>deposit()</i>	Partial; use Curve ETHx/ETH	—
Mantle	mETH	Mantle, ETH	mETHStaking. <i>stake()</i>	Partial	Native to Mantle chain
Ether.fi	eETH / weETH	ETH, Base, Arb	LiquidityPool. <i>deposit()</i> ; wrap eETH→weETH	Patchy; aggregator misses depending on version	Restake-capable (LRT-adjacent)
Renzo	ezETH	ETH, Arb, Linea, Base	RestakeManager. <i>depositETH()</i>	Rarely indexed by aggregators	Use Balancer ezETH/wETH as secondary
Kelp	rsETH	ETH, Arb, Optimism	LRTDepositPool. <i>depositETH()</i>	Rarely indexed	Secondary via Curve rsETH/wETH
Swell	rswETH (LRT) / swETH (LST)	ETH	RswETH. <i>deposit</i> / SwETH. <i>deposit</i> payable	Partial	swETH = LST, rswETH = LRT
Puffer	pufETH	ETH	PufferDepositor. <i>depositETH</i>	Rarely indexed	Secondary via Pendle / Curve
EigenLayer LRT restake	Underlying-LST restaked	ETH	Per-strategy <i>deposit</i> calls	Not aggregator-routable for restaking leg	Restake is its own protocol step

Resolution strategy when aggregator doesn't index an LST/LRT.

- Builder maintains a per-LST/LRT *direct_mint_path* registry: protocol address + method selector + ABI shape.
- When user requests “add ETH liquidity using ezETH”, the planner emits an intent with *source_token*: *ezETH*. The Resolver looks up ezETH → finds it's not in any aggregator's token list. The Builder falls back to (a) *secondary market swap* (e.g., Balancer ezETH/wETH pool) if liquid enough, OR (b) *direct mint* if the user explicitly wants new issuance (some users prefer fresh restake exposure).
- Always surface in Preview which path was chosen: “Routing via secondary market (Balancer ezETH/wETH 5bp, 8M TVL) — saves 12 bp vs direct mint”.
- Cap-aware: if a direct-mint protocol is at cap (Rocket Pool routinely is), automatically fall to secondary without asking.

10. Stub Families (delegate-to-aggregator)

These protocol families are supported via aggregator-delegation. No native deposit IX is built; the assistant routes through Enso (EVM) or Jupiter (Solana). Range is N/A. Receipt verification falls back to wallet-balance delta.

Family	Chain(s)	Routing strategy
Uniswap V2 forks (Sushi V2, PancakeSwap V2, Quickswap V2, Camelot V2, Velodrome V1, Aerodrome V1, Baseswap, Trader Joe V1, Spookyswap, Spiritswap)	EVM	Delegate to Enso /shortcuts/route. Zap-in via Enso's built-in 50/50 split. Receipt is ERC-20 LP. No range. Approve token0 + token1 OR rely on Enso's Permit2 path.
V3 forks not in core support (SushiSwap V3, Camelot V3, Ramses CL, Quickswap V3, Thena Fusion, KodiakV3)	EVM (per fork)	Several use the Algebra Integral backend, not Uniswap V3 — Algebra has dynamic fee tiers and a different NFPM. For 2026 launch, delegate Algebra-backed forks to Enso entirely (it has Algebra coverage); plan native support only if a single fork accrues > 5% of Ilyon LP volume.
Maverick V2	ETH, Base, BSC, Sonic	Binned AMM with four LP modes (Static, Right, Left, Both — Right and Left auto-follow price). Native SDK exists (@maverickprotocol/maverick-sdk) but coverage is narrow. Stub for v1; revisit if Base TVL grows. Receipt: ERC-20 LP.
Trader Joe V2 / V2.1 / V2.2 (LB pools)	Avalanche, Arb, BSC	Liquidity Book — bin-based discrete liquidity (similar to DLMM). Multiple LP modes (spot, curve, bid-ask). Native SDK exists but launch volumes on Ilyon's priority chains are modest. Stub: delegate to Enso for Avax + Arb where available.
Lifinity (proactive market making)	Solana	Oracle-driven AMM (does not provide passive LP positions in the traditional sense). Not LP-add-target for retail users in 2026. Stub.
Cropper, Aldrin, Crema	Solana	Long-tail Solana DEXes. Delegate to Jupiter for swap routing; native LP add not supported. Stub.
Phoenix CLOB, Openbook V2	Solana	Central limit order books — not pools. Out of scope for an LP-execution surface. (Listed for completeness; resolver should refuse with explanation if user asks.)
Save (formerly Solend)	Solana	Lending market. Single-sided supply. Delegate to Jupiter or direct Save SDK for the deposit leg; no range. Stub for v1, native in v2 if user demand justifies.
Drift super-stakeSOL, Lulo, MarginFi	Solana	Lending markets with specialized features (Drift super-stake = looped JitoSOL/mSOL). Stub; complex risk profile warrants explicit native support before promotion. Delegate-mode permitted for read-only portfolio display.
Trident	Solana	AMM family with several pool types (constant-product, constant-sum, concentrated, weighted). Native SDK exists but TVL is small. Stub.

11. Technical-Surface Recommendations & Trade-offs

For each functional concern, the recommended primary tool plus the alternatives weighed against.

Concern	Primary	Alternatives weighed	When primary doesn't fit
EVM swap router (general)	Enso / shortcuts/route as catch-all (position-token aware, multi-input bundling, supports zaps into LPs).	1inch v6 for best-price simple swaps (no LP-position awareness). 0x Swap API for institutional reliability but fewer protocols. KyberSwap Aggregator for stable-pair tight routes. Odos best for Curve-heavy routes. Paraswap for legacy compatibility on chains 1inch lags. Native protocol routers (Uniswap Universal Router, PancakeSwap Smart Router) for in-protocol-only routes (cheaper, no aggregator fee).	Use 1inch v6 for pre-swap when LP is in a Uniswap V3-class pool and price impact > 30 bps — its routing beats Enso on raw swap quality. Use Enso when the LP target is anywhere Enso has a native shortcut (better atomicity).
EVM LP-position calldata (V3-class)	@uniswap/v3-sdk + v3-sdk-extended for math; NFPM ABI direct for calldata.	Enso has v3 LP coverage but its position-token outputs are not always indexable in Ilyon's position-tracking schema. Native SDK gives more control.	Native SDK for v1; Enso fallback for chains the SDK doesn't cover (Sonic, Berachain, Unichain at launch).
EVM V4 calldata	@uniswap/v4-sdk (V4Planner / Actions encoder).	No real alternative — V4 is too new for aggregators to handle position mints reliably.	Permit2 mandatory. Hook allowlist before any V4 plan goes to Shield.
Solana swap router	Jupiter Swap API (default) and Jupiter Ultra (premium).	Direct AMM SDKs (Raydium, Orca, Meteora) for in-protocol routes; rarely needed since Jupiter aggregates them all.	Use Jupiter Ultra (with its RFQ + MEV protection + Jito bundles) for swaps > \$5k OR pairs with known sandwich exposure. Default to Swap API for < \$5k retail flow. Jupiter Trigger is for stop / limit orders — not relevant to LP execution.
Solana LP-position calldata	Native protocol SDKs only: @orca-so/whirlpools, @raydium-io/raydium-sdk-v2, @meteora-ag/dlmm.	No Solana aggregator builds LP-position instructions. Jupiter does swaps only.	Mandatory.
Cross-chain bridging	deBridge DLN (intent-based, solver-filled, snapshot-able quote, fast 30–90s).	LI.FI as fallback (aggregator over Across, Stargate, Hop, etc.) — better when DLN doesn't cover the corridor. Socket / Bungee similar to LI.FI. Across raw is fast and cheap on EVM-to-EVM but doesn't cover Solana. Squid for Cosmos-adjacent (out of scope but worth noting).	DLN is the right default because the solver model gives us a snapshot quote for the snapshot/rebuild pattern (§6c). Native bridges (Across burn-and-mint raw) are forbidden by spec unless explicitly justified.
EVM simulation	Tenderly bundle simulation for production preview.	eth_call + state-override via own RPC for cost-sensitive paths (state-overrides let you simulate without spending tx-budget). Phalcon / blocksec.com for adversarial simulation (when Shield flags MEV-rich txs).	Tenderly for production; eth_call+override for hot paths.

Concern	Primary	Alternatives weighed	When primary doesn't fit
Solana simulation	simulateTransaction with <code>replaceRecentBlockhash + sigVerify=false</code> .	No mainstream third-party Solana simulator.	Always include token-program reads to assert balance deltas.
MEV protection (EVM)	MEVBlocker as default private mempool (zero cost). Flashbots Protect when user is on Ethereum mainnet and explicitly wants Flashbots. CoW Hooks if the pre-swap leg can be intent-batched with CoW.	BloXroute private mempool — strong but commercial. Eden deprecated. 1inch Fusion / Fusion+ when using 1inch as router (built-in MEV protection).	MEVBlocker by default for any swap > \$5k or any tx with > 30 bps expected price impact. Always private for LP rebalances near boundary (JIT-attack risk).
Position indexing	Subgraphs per protocol (The Graph) for EVM. Helius parsed-instruction stream + Birdeye position API for Solana.	Moralis for portfolio-level rollups. DexScreener for pool-level discovery.	Subgraphs are authoritative; Moralis is the consolidation layer.
Approval handling	Permit2 wherever supported (Uniswap V3/V4, Balancer V3, UniswapX, Universal Router).	EIP-2612 permit for individual ERC-20s that support it (USDC, DAI, USDT-on-some-chains).	Never unlimited approve; always scope to exact-amount + short-lived (Permit2 expiry default 24 hours).

12. Normalized JSON Schema — DepositPlan

The canonical in-chat “pool deposit plan” object. Every combination from Chapters 7–10 must be expressible in this schema. Designed for: (a) full audit/replay, (b) plan rebuilding under async-fill, (c) per-step recovery, (d) chat re-rendering at any state.

```
DepositPlan { plan_id: UUID, created_at: int64 (ms), expires_at: int64 (ms), // simulation
freshness; refresh required past this source: { raw_prompt: string, parsed_at: int64, model:
string, model_version: string, confidence: float }, intent: LiquidityIntent { action:
enum(ADD, INCREASE, DECREASE, COLLECT, REBALANCE, CLOSE, ZAP_IN, ZAP_OUT, MIGRATE),
user_wallet: { evm_addr?:, solana_pubkey? }, chain_id: int, // dst chain (for cross-chain, src
in cross_chain.src_chain) protocol: enum(...), // see §8 matrix protocol_version: enum(V1, V2,
V3, V4, NG, CLMM, DLMM, CPMM), pool_descriptor: { pool_address?: Address, // EVM; null for V4
(use poolKey_hash) poolKey_hash?: bytes32, // Uniswap V4 pool_pubkey?: Pubkey, // Solana
token0: TokenRef, token1: TokenRef, fee_bps?: int, tick_spacing?: int, // Slipstream/V4/V3
bin_step_bps?: int, // DLMM/Maverick hooks?: Address, // V4 verified: bool // Shield allowlist
}, amounts: { mode: enum(EXACT_TOKEN0, EXACT_TOKEN1, EXACT_BOTH, EXACT_USD, PROPORTIONAL,
PERCENT_OF_POSITION, ALL), token0_amount?: Decimal, token1_amount?: Decimal, usd_amount?:
Decimal, percent?: int }, range?: { mode: enum(FULL, WIDE, BALANCED, TIGHT, CUSTOM_TICKS,
CUSTOM_PRICE), lower_tick?: int, // V3-class / Whirlpool / Raydium CLMM upper_tick?: int,
lower_bin?: int, // DLMM upper_bin?: int, lower_price?: Decimal, // user-facing (always
round-tripped to ticks) upper_price?: Decimal, preset_label?: string, rounded_to_valid_ticks:
bool, capital_efficiency_x: float, in_range_probability_30d: float }, strategy?: { // DLMM /
Maverick distribution: enum(SPOT, CURVE, BID_ASK, MAVERICK_STATIC, MAVERICK_RIGHT,
MAVERICK_LEFT, MAVERICK_BOTH) }, execution: { slippage_bps: int, // default 50
deadline_seconds: int, // default 600 mev_protection: enum(NONE, MEVBLOCKER, FLASHBOTS,
COWHOOKS, PRIVATE_RPC, JITO_BUNDLE), gas_strategy: enum(NORMAL, FAST, INSTANT), stake_rewards:
bool, use_permit2: bool, auto_unwrap_native_on_remainder: bool }, cross_chain?: {
src_chain_id: int, src_token: Address, bridge: enum(DEBRIDGE_DLN, LIFI, SOCKET, ACROSS),
snapshot_dst_amount: Decimal, solver_slippage_bps_band: { min: int, max: int }, order_id?:
string, fill_status: enum(PENDING, FILLED, FAILED, REFUNDED), pending_dst_fill: bool },
funding_plan: { user_balances_snapshot: [{ chain_id, token, amount, usd_value }, ...],
source_tokens: [{ chain_id, token, amount }, ...], swap_legs: [ { leg_id, chain_id,
from_token, to_token, amount_in, expected_out, slippage_bps_budget, router: enum(...),
provenance_url?, dust_accumulated } ], bridge_legs: [ { leg_id, src_chain_id, dst_chain_id,
src_token, dst_token, amount_in, snapshot_expected_amount, bridge: enum(...),
pending_dst_fill: bool, fill_resolved_amount?: Decimal, solver_slippage_realized_bps?: int }
], dust_accumulated_total_usd: Decimal, dust_disposition: enum(KEEP_IN_WALLET,
SWEEP_INTO_POSITION, INCREASE_LATER) }, range_quotation: { // null for non-CL families
apr_curve_30d: [{ range_width_pct, apr_fee, apr_reward, apr_total, in_range_prob }, ...],
expected_il_table: [ { price_move_pct: int, hodl_value_usd, lp_value_usd, il_pct,
includes_fees: bool } ], expected_apr_at_selected_range: { fee, reward, total }, data_sources:
[ string ] // “DeFiLlama 30d”, “Birdeye OHLCV”, ... }, steps: [ // ordered, dependency-aware {
step_id: int, kind: enum(APPROVE, PERMIT2_SIG, SWAP, BRIDGE, WRAP_NATIVE, UNWRAP_NATIVE,
ATA_CREATE, TICK_ARRAY_INIT, BIN_ARRAY_INIT, MINT_POSITION, INCREASE_LIQUIDITY, DEPOSIT,
STAKE, COMPOUND, MULTICALL), chain_id: int, depends_on: [ step_id ], blockers: [
enum(PENDING_DST_FILL, ADAPTER_QUOTE_REQUIRED, PRICE_DRIFT_RESIMULATE, APPROVAL_MISSING,
ATA_CREATION_MISSING, PENDING_EPOCH_ENTRY) ], tx_skeleton: { // calldata description, no bytes
target_contract: Address | Pubkey, method: string, param_summary: { ... } // human-readable
for chat display }, gas_estimate: { native, usd }, simulated_diff: { in: [...], out: [...] },
expected_duration_ms: int, recovery_hook: { on_failure: enum(AUTO_REBUILD, ASK_USER,
REFUND_SWAP_BACK, ESCALATE_NOTIFY), max_auto_retries: int }, provenance: { source: enum(ENS0,
ONEINCH, JUPITER, NATIVE_SDK, ...), quote_id: string } } ], expected_outputs: { receipt_kind:
enum(LP_ERC20, BPT, V3_NFT, V4_NFT, ATOKEN, ERC4626_SHARE, KTOKEN, POSITION_PDA,
POSITION_PDA_WITH_NFT, LP_MINT_SPL, JLP, MSOL, JITOSOL, INF, LST_ERC20), receipt_address?:
Address | Pubkey, expected_amount?: Decimal, expected_token_id?: uint256, // for NFTs (only
known post-mint) verification_rpc_read: string // human description of the verification check
}, shield_verdict: { pool_allowlist_status: enum(ALLOWED, EXPERIMENTAL, BLOCKED), token0_risk:
{ score, flags: [...] }, token1_risk: { score, flags: [...] }, hook_risk?: { allowlist_match:
bool, audited: bool, flags: [...] }, mev_exposure_flag: bool, jit_attack_proximity: enum(NONE,
LOW, MEDIUM, HIGH), sanctions_clean: bool }, state: enum(DRAFTING, READY_TO_SIGN,
```

```
SIGNING_IN_PROGRESS, BROADCASTED, PENDING_DST_FILL, REBUILDING, CONFIRMING, VERIFYING,  
INDEXED, FAILED, STUCK_BALANCE, CANCELLED) }
```

13. Edge-Case Appendix

Every edge case the implementation must handle, with the recommended behavior. Sorted by frequency / severity.

Edge case	Recommended behavior
Stale price feed	If Pyth / Chainlink price > 60s old when computing range or slippage caps, refuse to build and ask user to retry. Never use stale prices for hard caps.
Decimal mismatch (USDC = 6 dec, but BSC USDC = 18 dec)	Resolver canonicalizes everything via on-chain <i>decimals()</i> per (chain, address). Never assume by symbol. Builder uses BigNumber arithmetic exclusively.
Checksummed vs lowercased addresses	Internal canonical form: lowercased hex. Display: EIP-55 checksummed. Comparisons always after toLowerCase().
Token-2022 transfer hook	Builder must check token program (TokenProgram vs Token-2022) and route through compatible IX variants. If pool doesn't support Token-2022 for that token, refuse with "pool does not accept this token-program variant" explanation.
Frozen account	Pre-flight: <i>account.is_frozen</i> check. If frozen, refuse with "your token account is frozen by the mint authority".
WSOL wrap/sync-native	Always include <i>sync_native</i> IX after <i>transfer SOL to WSOL</i> ATA. Always <i>close_account</i> on the WSOL ATA after deposit to reclaim rent (if user didn't deposit all SOL).
Native ETH wrap (V3 / older)	V3 needs WETH; V4 supports native. Builder always checks pool currency and wraps only when needed.
Pool requires exact ratio (no zap path)	If a pool is two-token-balanced (Curve plain, Balancer weighted) and user has only one token, route through (a) pool's single-coin add if supported (Curve), (b) Balancer single-token-exact-out, or (c) pre-swap to balance. Show ratio in Preview.
Deposit caps	Aave V3, Pendle markets, JLP, klend reserves all enforce caps. Always check; refuse with "cap reached, try smaller size or alternative pool".
Allowlist / KYC gates	Maple, Goldfinch, some Balancer Boosted pools require KYC or whitelisting. Refuse with "this pool requires off-chain verification; not supported in chat". Surface as non-recoverable.
Epoch-locked entry	Pendle PT/YT mint near maturity, Curve gauge boost checkpoint, Marinade epoch activation. Block with <i>pending_epoch_entry</i> + ETA.
Multi-reward APR pricing	Some pools emit 3+ reward tokens (Slipstream + Merkl). Each priced via spot oracle. Surface as "Fee APR 8.2% + AERO 6.1% + (Merkl) ARB 2.4% = 16.7% total".
Aggregator outage	Each external service has a circuit breaker on 3-of-5 consecutive failures. Fall to next-best (Enso → 1inch → 0x → KyberSwap). Surface degradation in chat: "Routing via 1inch — Enso is currently slow".
Wrong wallet for chain	Solana wallet (Phantom) on EVM intent → Phantom does sign EVM if enabled; otherwise ask for MetaMask. Solflare-only on EVM intent → refuse and ask user to add an EVM wallet. Detect early at planning, not at sign time.
Hardware wallet outer-instruction limit	Ledger fits ~7 outer instructions in a Solana tx; Phantom mobile rejects oversized versioned tx (>1232 bytes without ALT, >~30 ALT keys). Builder splits into multiple signed txs when limits exceeded; chat shows progress.
Sandwich / MEV exposure on swap legs	If estimated price impact > 30 bps OR notional > \$5k, force MEVBlocker (EVM) or Jito bundle (Solana) regardless of user toggle. Surface as "MEV protection auto-enabled".
JIT-attack adjacency	Shield monitors pending mempool for large swaps in pools the user is about to LP into. If a large swap is queued, delay user's LP by 1 block and re-simulate (averts JIT fee-dilution; 85% dilution figure per Xiong et al. 2023).

Edge case	Recommended behavior
Sim-pass / exec-fail	Slippage gap: re-simulate with current state; if >50 bps drift, propose wider slippage. Blockhash expiry (Solana): refresh blockhash and retry. Base-fee spike (EVM): bump maxFeePerGas, ask user to confirm.
Tick-spacing constraint	Always round to nearest valid ($\text{tickLower} - \text{tickLower} \% \text{tickSpacing}$, $\text{tickUpper} + (\text{tickSpacing} - \text{tickUpper} \% \text{tickSpacing}) \% \text{tickSpacing}$). Show rounded values in Preview with the original-vs-rounded delta.
Permit2 vs approve fallback	If wallet doesn't support Permit2 (older versions), fall back to ERC-20 approve. Surface as "two signatures required (your wallet doesn't support Permit2)".
EIP-1559 vs legacy gas	BSC and some L2s default to legacy. Auto-detect via chain support and set tx type accordingly. Always show effective gas price in Preview.
Nonce management	If user has pending txs on the chain, builder uses the next available nonce. Surface "you have a pending tx from earlier — your new LP will go after it confirms".
Gas-token availability	Pre-flight: balance check for gas token on dst chain. If < 2× estimated gas, offer to bridge a small gas top-up first. Auto-bundle if user opts in.
Aggregator returns null route	Token pair has no liquidity path. Refuse with "no route found from {src} to {dst} on {chain}; try a different source token or chain".
Pool not initialized	V4 pools / Whirlpools / Raydium CLMM all require <i>initialize</i> before first deposit. If pool exists but is uninitialized, offer to initialize (warn that the user becomes the de-facto price-setter — high risk if not already aware of fair price).
Self-trade against own LP	If user's pre-swap leg trades against the very pool they're about to LP into, it self-impacts price + costs the user the spread. Builder detects this and reroutes the pre-swap through a different pool.
Token approval to wrong spender	Long-standing common bug: approving the wrong router. Builder asserts spender matches the router being used in the next step; if mismatch, refuses.

14. Open Design Questions

Items that warrant a design review before code. Each has a recommended default plus the trade-off to weigh.

Question	Recommended default	Reasoning
Pool allowlist strictness	Strict (top-N per chain per protocol) at launch, expandable by signed reputation data.	Strict trades coverage for safety. Reputation damage from a single rug-via-Ilyon is asymmetric; safer to start strict and grow the list with telemetry.
Permit2-only EVM approvals	Yes — default to Permit2, fall back to ERC-20 approve only when the wallet doesn't support it.	MetaMask, Phantom-EVM, Rabby, Rainbow, OKX all support Permit2 as of 2026. Holdouts are older WalletConnect-only wallets.
New-pool creation in chat	Refuse in v1. Direct users to protocol UI for first-deposit / pool-init.	Pool init is the highest-risk parameter surface (wrong price, wrong tickSpacing). Not worth the surface area in v1.
Session-key autonomy (v2 only, post-EIP-7702)	Per-policy expiry + cumulative spendingLimit + one-tap revoke-all. On-chain enforcement.	Off-chain-only policy is unacceptable because Ilyon's own service is the attacker model. Always on-chain.
Disclaimer copy and surface	Persistent "AI can make mistakes — review every transaction" near every Confirm button. Yield numbers always labeled as projections with source. No unsolicited personalized recommendations.	Legal-review item, not engineering, but gates launch. Keeps Ilyon outside SEC/CFTC investment-adviser perimeter.
Cross-chain composed plan UX	Single chat thread with three steps shown upfront; step 3 greyed until fill.	Two-separate-plans is a regression. Confirmed in §6c.
"With my USDT" default behavior	Smart heuristic (§6d): prefer USDT-native alternative if comparable; else clarifier; else split-zap with explicit ratio + exposure disclosure.	Pure silent-split is mechanically correct but quietly increases user exposure.
Real-data APR curve refresh cadence	Pool Index daily; APR curve per-pool hourly; range-specific computation on demand (cached 5 min).	Hourly is fast enough; daily on pool index reduces subgraph load.
Indexer alerting channels in v1	In-app only at launch. Add Telegram + email in v1.1.	Telegram requires bot infra; email needs deliverability work. Defer.

How to use this spec

Once these defaults are agreed in a design review, the rest of the implementation is deterministic. The schema in Chapter 12 is the contract; the matrices in Chapters 7, 8, and 13 are the test set; the deep dives in Chapter 9 are the per-family build sheet; the seven head-on issues in Chapter 6 are the unblockers for the specific bugs IlyonAi is hitting today.

End of specification.